COPYRIGHT WARNING: Copyright in these original lectures is owned by Monash University. You may transcribe, take notes, download or stream lectures for the purpose of your research and study only. If used for any other purpose, (excluding exceptions in the Copyright Act 1969 (Cth)) the University may take legal action for infringement of copyright.

Do not share, redistribute, or upload the lecture to a third party without a written permission!

FIT3181/5215 Deep Learning

Week 06:Advanced Convolutional Neural Networks

CE/Lecturer: Dr Trung Le (Clayton)

Lecturer: Dr Binh Nguyen (Clayton)

Lecturer: Dr Ting Fung Fung (Malaysia)

Lecturer: Dr Arghya Pal (Malaysia)

Lecturer: Dr Sailaja Rajanala (Malaysia)

Email: trunglm@monash.edu

GROUP OF EIGHT AUSTRALIA

Department of Data Science and AI

Faculty of Information Technology, Monash University, Australia



Outline

- Deep learning before and after 2012
- Revision of building-blocks, rationales and drawbacks of CNNs
- SOTA Convolutional Neural Networks
- Visualization in CNNs
- Adversarial Machine Learning

- Further Reading Recommendation
  - [FDL, ch05], [HandsOn-2nd, ch14], [Dive Into Deep Learning, ch07]
  - "CS231n: Convolutional Neural Networks for Visual Recognition", Stanford University, 2016

Deep Learning before and after 2012

Deep learning renaissance

Before 2012

- Some **milestones** before 2012
  - Backprop was proposed in 1986
  - CNN was proposed in 1995
- People **did not believe** that NNs work
  - Could not train NNs for real-world datasets
  - Easy to get overfitting
- In machine learning (computer vision), **hand-crafted feature approach** dominated
  - SIFT (scale-invariant feature transform), SURF (speeded up robust features)

After 2012

- **AlexNet** won **ImageNet competition**
- **Missing Ingredient:** Data
  - In 2009, **the ImageNet dataset** was released, challenging researchers to learn models **from 1 million examples, 1000 each** from **1000 distinct categories** of objects
- **Missing Ingredient:** Hardware
  - **Graphical processing units (GPUs)** proved to be a game changer in making deep learning feasible.
  - These chips had long been developed to **optimize** for high throughput **4×4 matrix-vector products**, which are needed for many computer graphics tasks and **calculate convolutional layers**
- The **renaissance** of Deep Learning

Hand-crafted feature

Visual data

Learning algorithm and feature extractor are disconnected !

- **Traditional approach** (hand-crafted feature learning)

Input

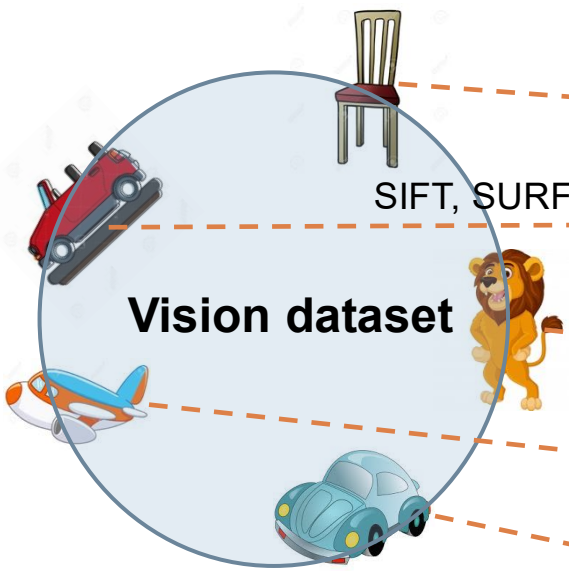


Feature extraction (SIFT, SURF, HOG)
Hand-crafted features



Learning algorithms (SVM, XGBoost)

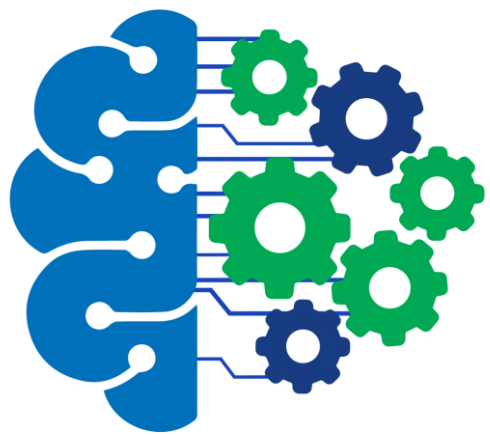




0.1	0.5	-1	2.1	1.3
0.6	0.2	1.3	2.5	3.3
0.7	0.5	-2	2.2	2.3
0.8	0.4	0.9	2.4	1.2
0.6	0.9	1.5	2.0	1.6

Feature vectors

[Source: Internet]



Prediction

ML model
(SVM, k-NN, decision tree, XG Boost)

Automatic feature extractor

Visual data

- **Deep learning approach** (automatic feature learning)

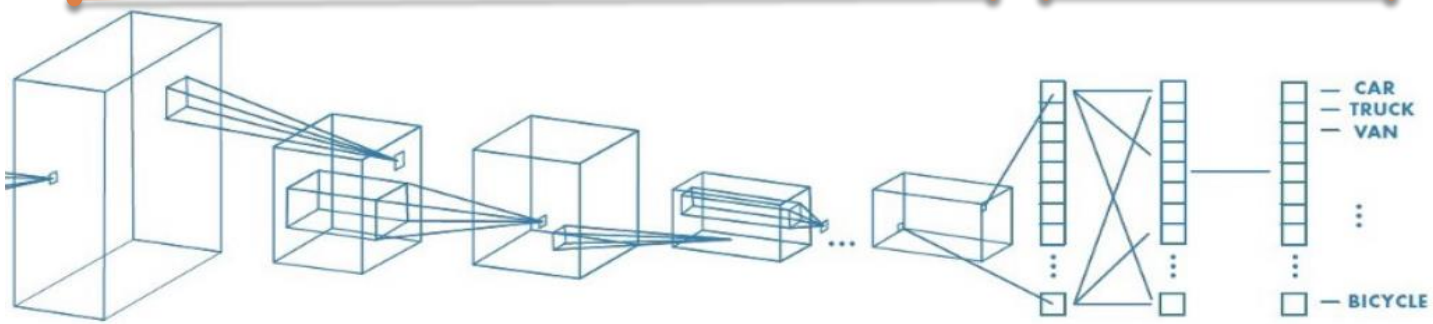
Classification and **feature extractor** are **connected** !

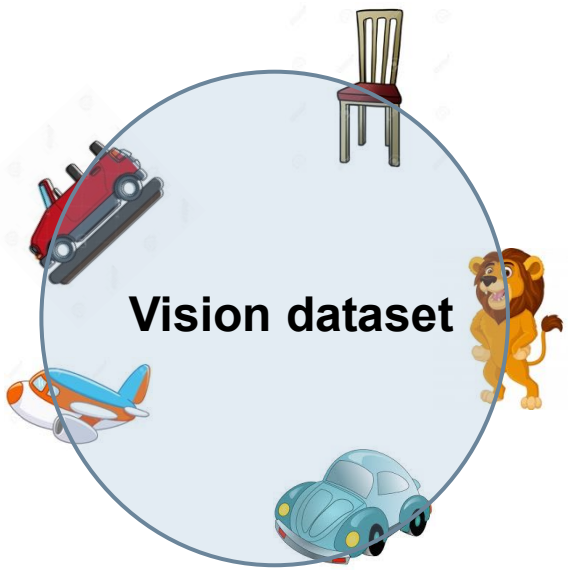
Input

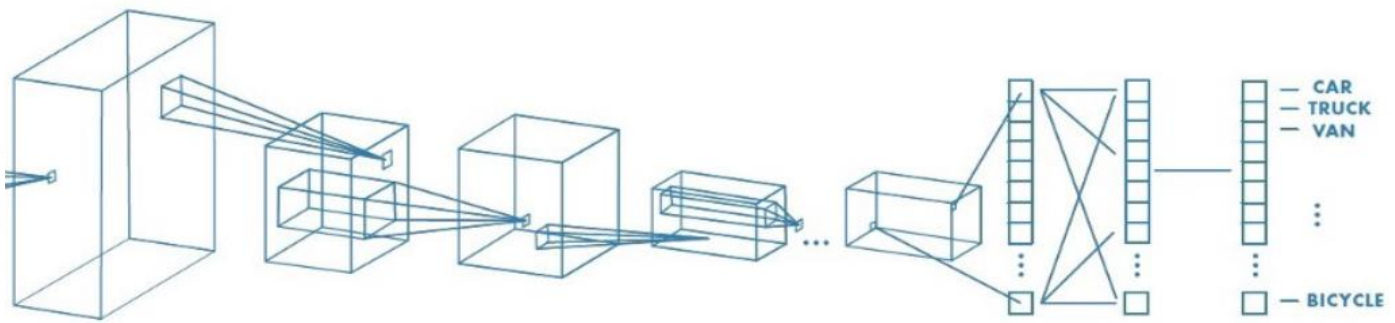
Feature extraction
Automatic features

Classification









Prediction

Hand-crafted and automatic feature extractor

Visual data

Learning algorithm and feature extractor are disconnected !

- **Traditional approach** (hand-crafted feature learning)

Input → **Feature extraction** (SIFT, SURF, HOG) → **Learning algorithms** (SVM, XGBoost)

Hand-crafted features







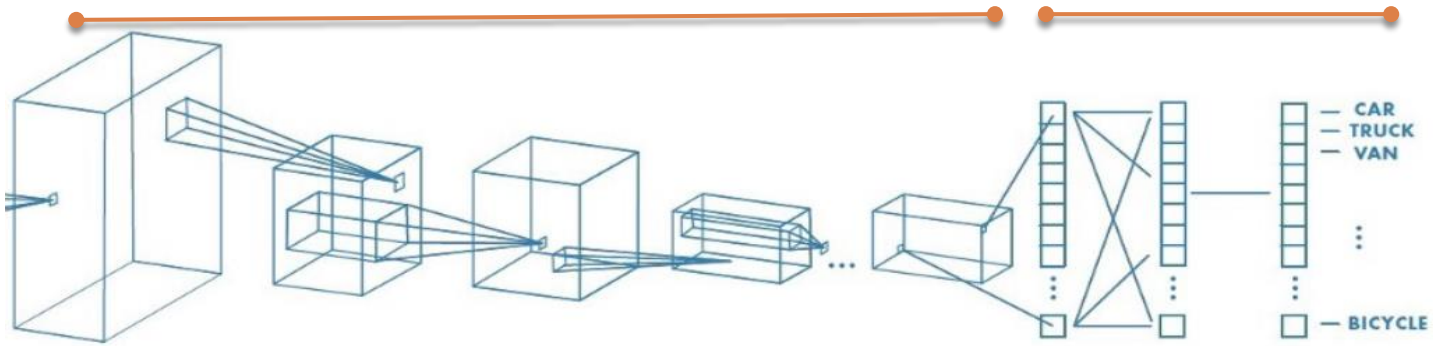
Classification and feature extractor are connected !

- **Deep learning approach** (automatic feature learning)

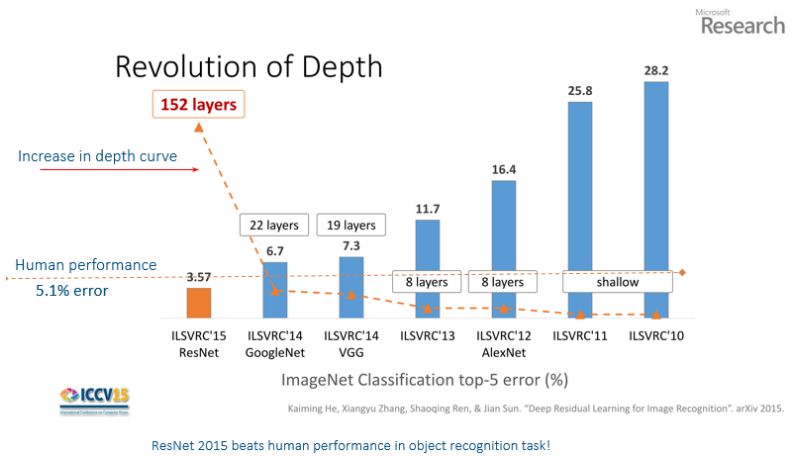
Input → **Feature extraction** → **Classification**

Automatic features



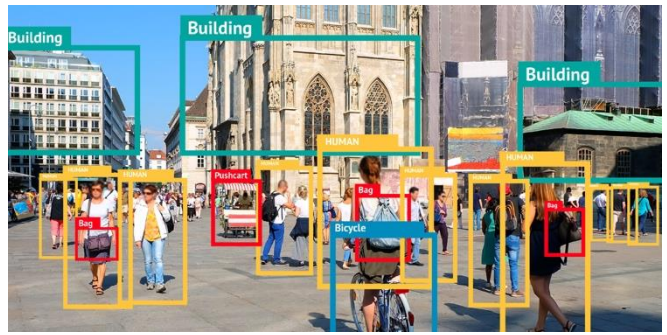


Achievements of CNNs



ResNet 2015 beats human performance in object recognition task!

Win human in object recognition on ImageNet dataset



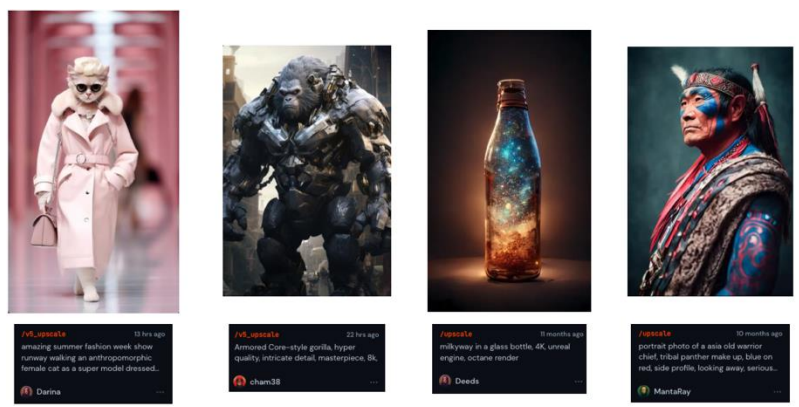


SOTA in object detection and semantic segmentation

Some fiction and imaginary tasks



Image captioning



Generative AI
DALL.E/Mid Journey

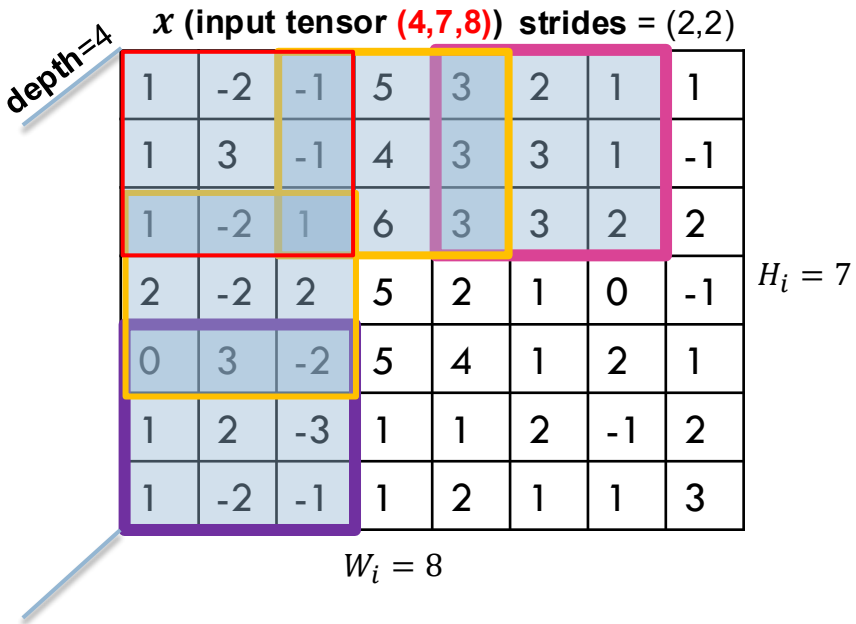


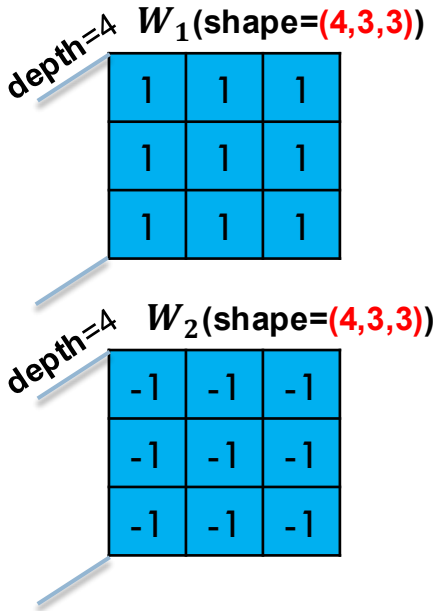
Tesla self-driving cars

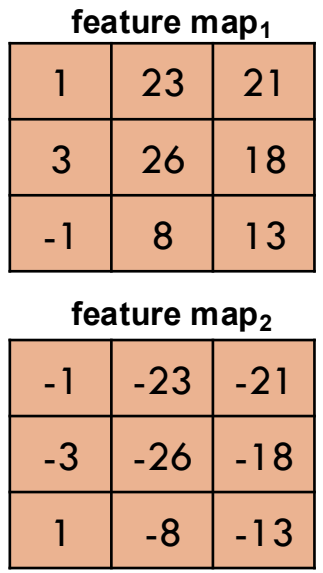


Revision of CNN building-blocks and How CNN works?

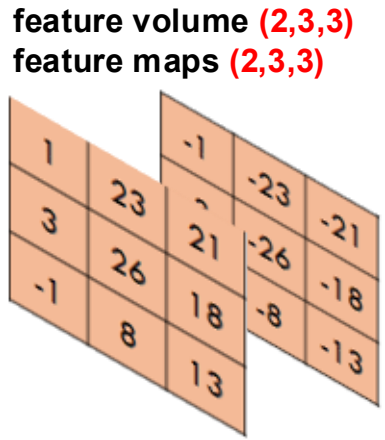
Convolution layer with multiple filters and feature maps



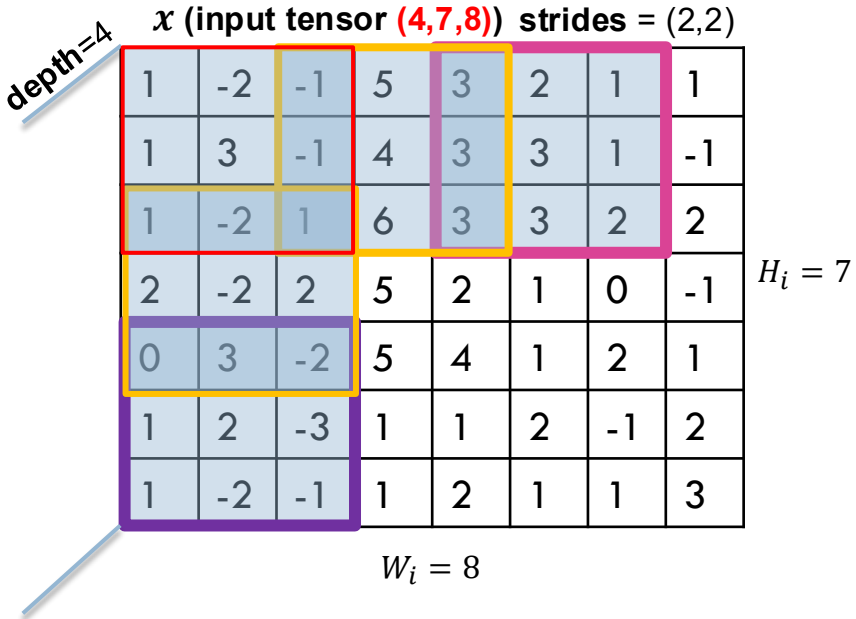


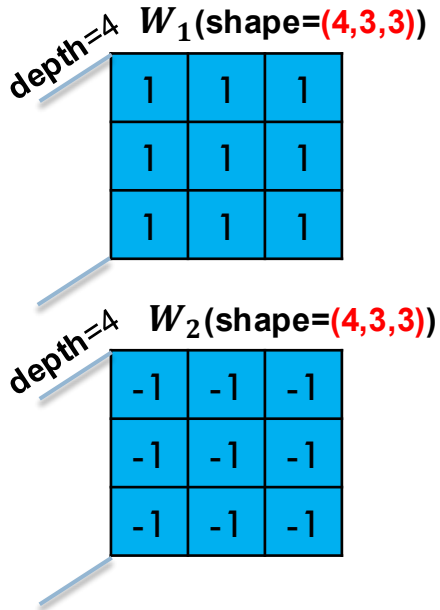


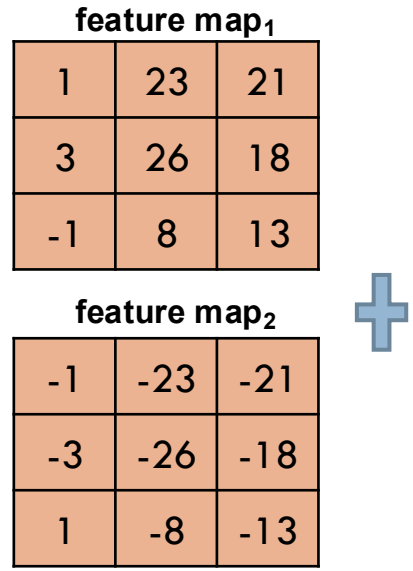


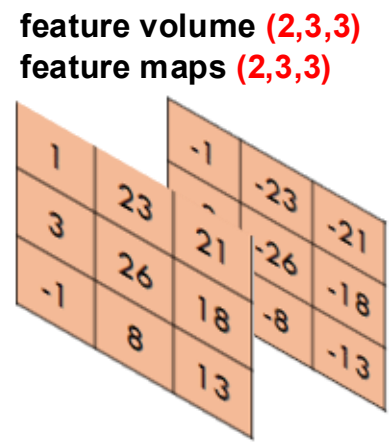


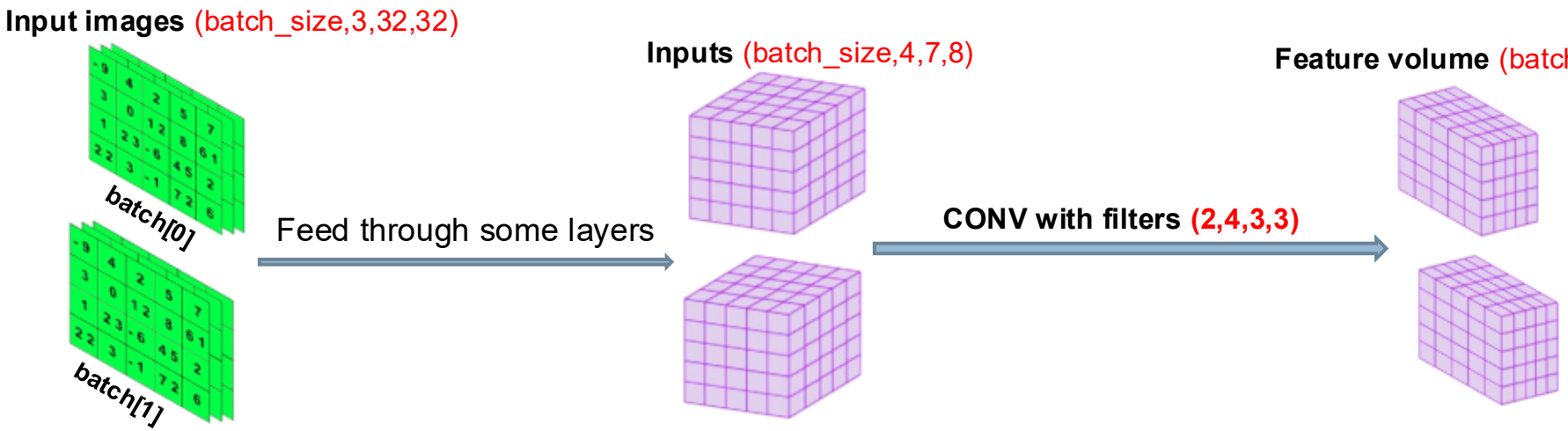
Convolution layer with multiple filters and feature maps









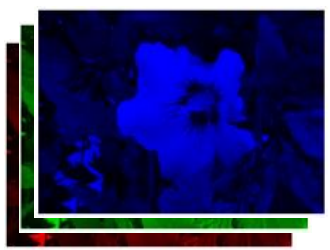



```
output = torch.nn.functional.conv2d(input = batch_tensor, weight= filters_tensor, stride =(2,2), padding= 3)
```

batch_size, in_channels , height, width,

out_channels, in_channels, fil_height, fil_width
,

CNN in Operation



[3,32,32]



Input Layer

Filters [4,3,4,4]

Filter 1

[3,4,4]

Filter 2

[3,4,4]

Filter 3

[3,4,4]

Filter 4

[3,4,4]

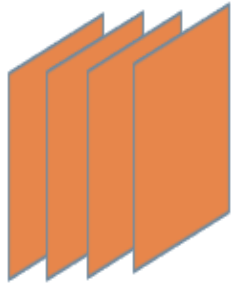
Conv2D 1

padding= 0
strides = (2,2)

Conv2d(3,4,kernel_size=4, stride=2, padding=0)

$$out_size = \left\lfloor \frac{32 + 0 - 4}{2} \right\rfloor + 1 = 15$$

Feature volume
Feature maps



[4,15,15]

Latent representation

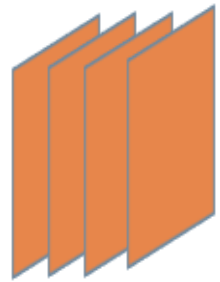
Pooling layer

pool-size=(2,2)
strides= (2,2)
padding= 1

MaxPool2d(kernel_size=2, stride=2, padding=1)

$$out_size = \left\lfloor \frac{15 + 2 \times 1 - 2}{2} \right\rfloor + 1 = 8$$

Feature volume
Feature maps



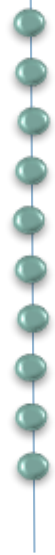
[4,8,8]

Latent representation

FC layer

4x8x8 neurons

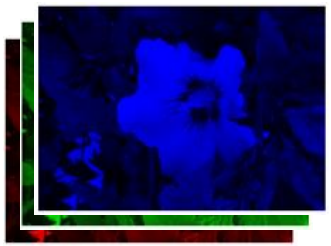
softmax



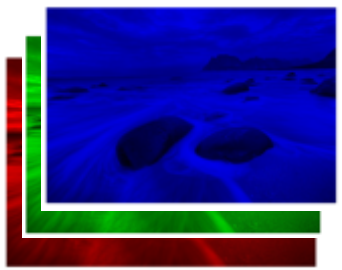
Output layer

10 neurons for 10 classes

CNN in Operation



[3,32,32]



[3,32,32]

[2,3,32,32]



Input Layer

Filters [4,3,4,4]

Filter 1

[3,4,4]

Filter 2

[3,4,4]

Filter 3

[3,4,4]

Filter 4

[3,4,4]

Conv2D 1

padding= 0
strides = (2,2)

Feature volume
Feature maps
[2,4,15,15]

[4,15,15]

[4,15,15]

Pooling layer

pool-size=(2,2)
strides= (2,2)
padding= 1

Feature volume
Feature maps
[2,4,8,8]

[4,8,8]

[4,8,8]

4x8x8

2

10

2

softmax

Output layer

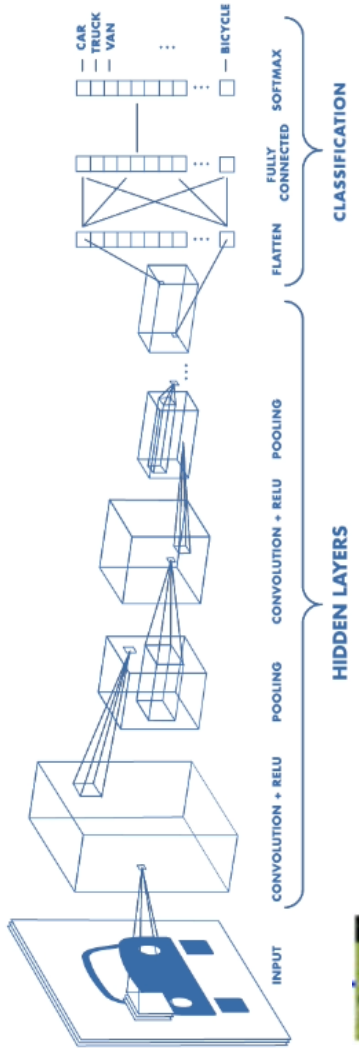
10 neurons
for 10 classes

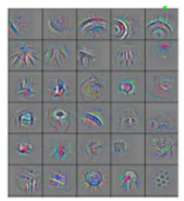
FC layer

4x8x8 neurons

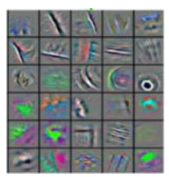
Automatic feature extractor

Deep learning for visual data





High-level feature/representation
- objects



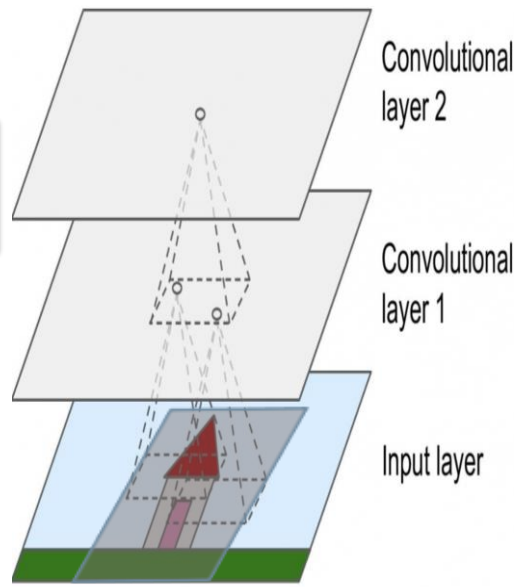
Mid-level feature/representation
- circles, triangles, boxes

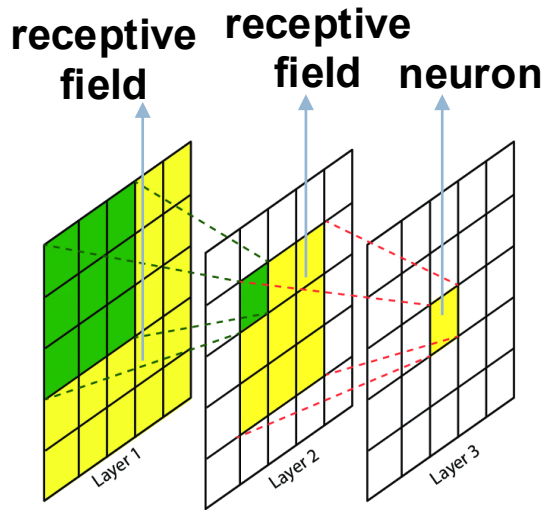


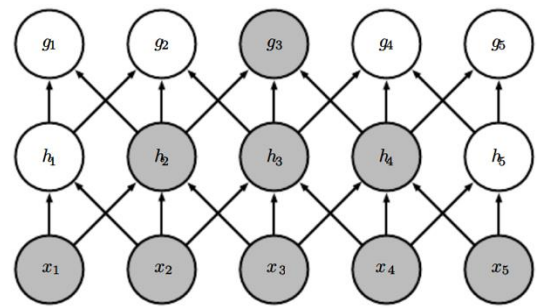
Low-level feature/representation
- edges, pixels, corners



Neurons on **higher layers** have **larger receptive fields** (patches) on **input images**

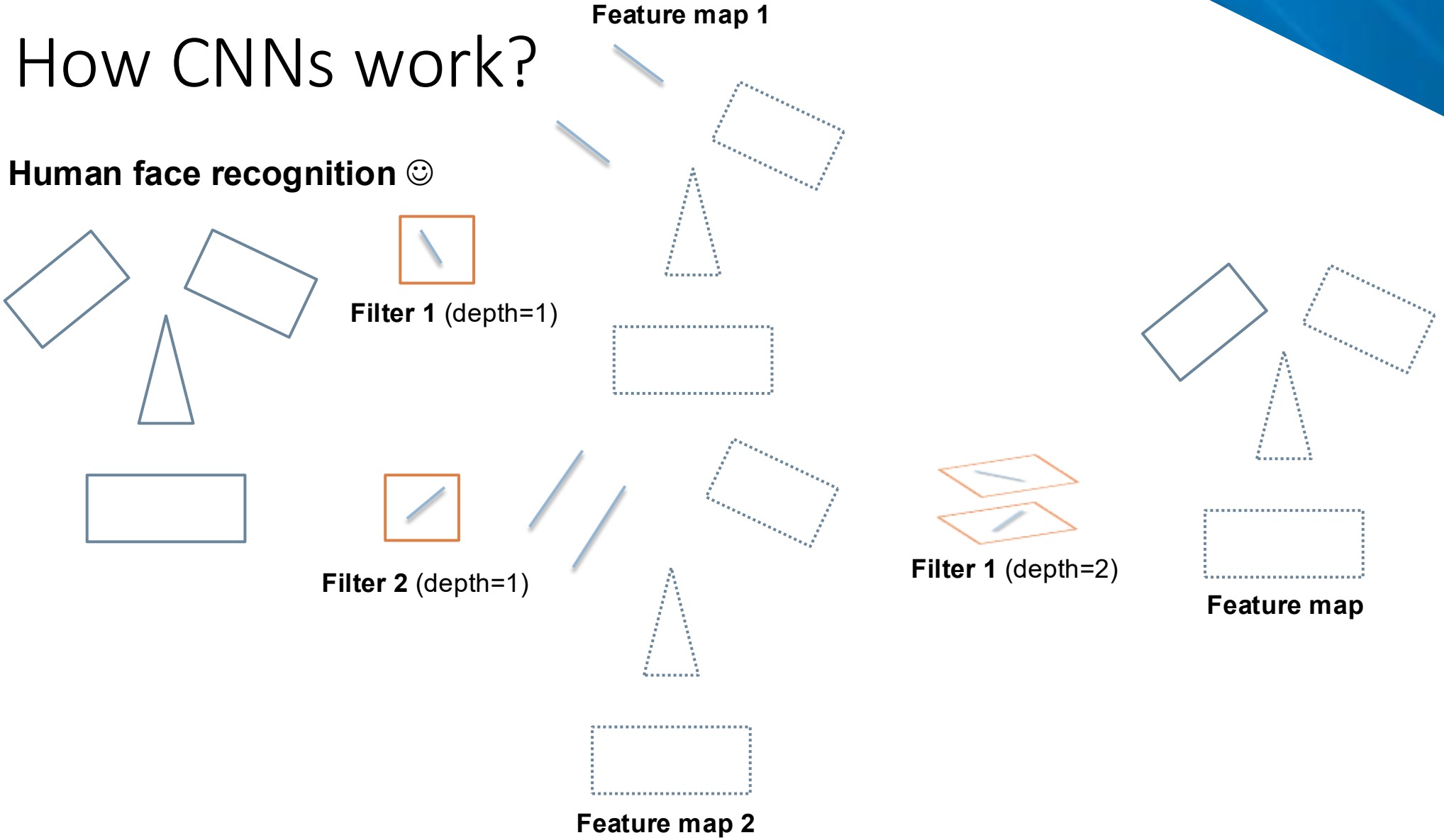






How CNNs work?

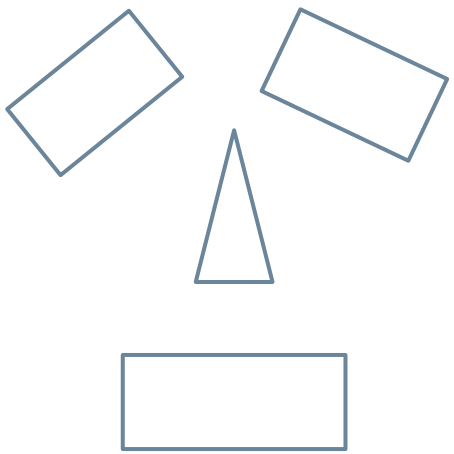
Human face recognition ☺



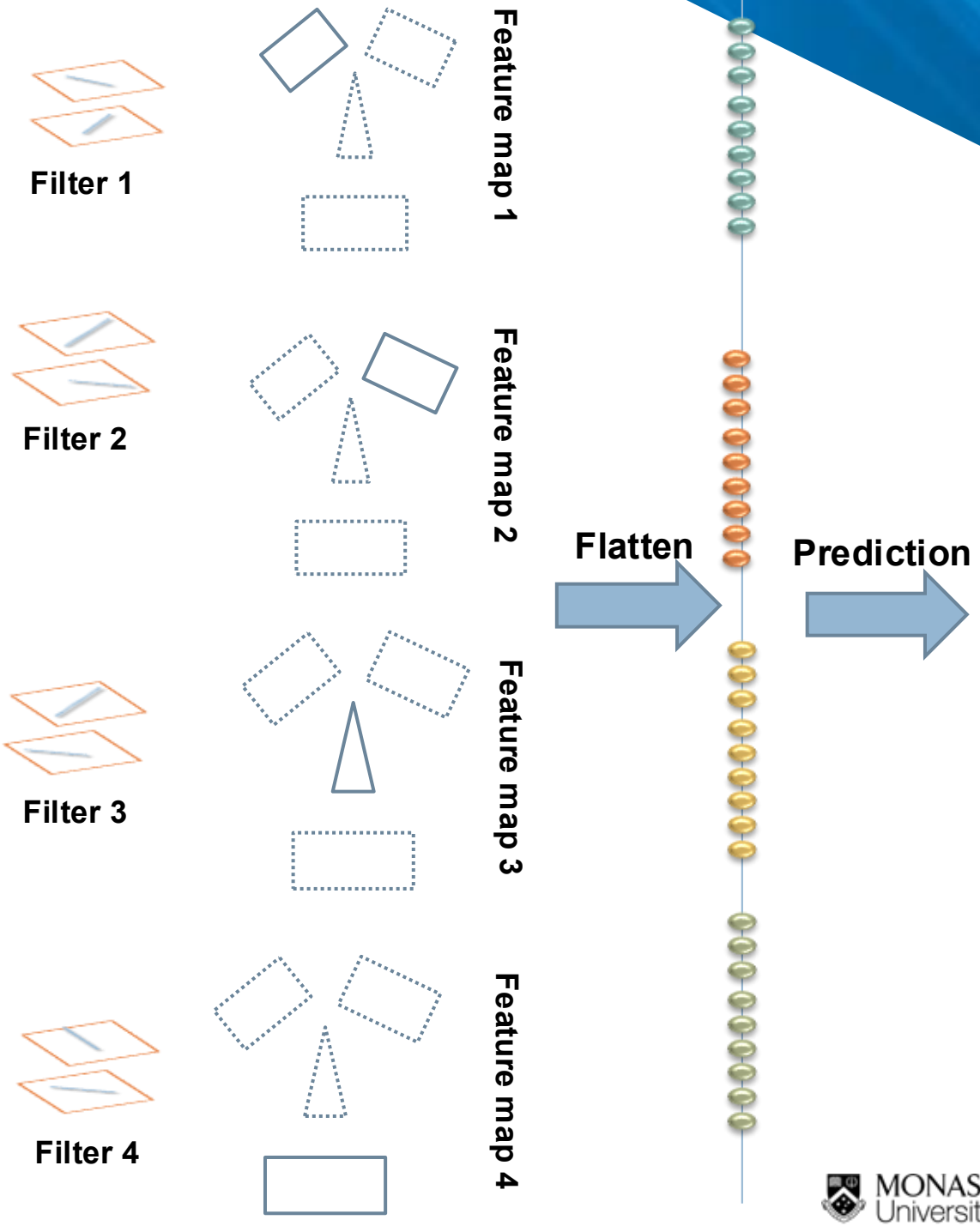
- Higher level filters **combine** lower level filters to represent **more abstract objects**
  - The context is **locally expanded**

How CNNs work?

Human face recognition ☺

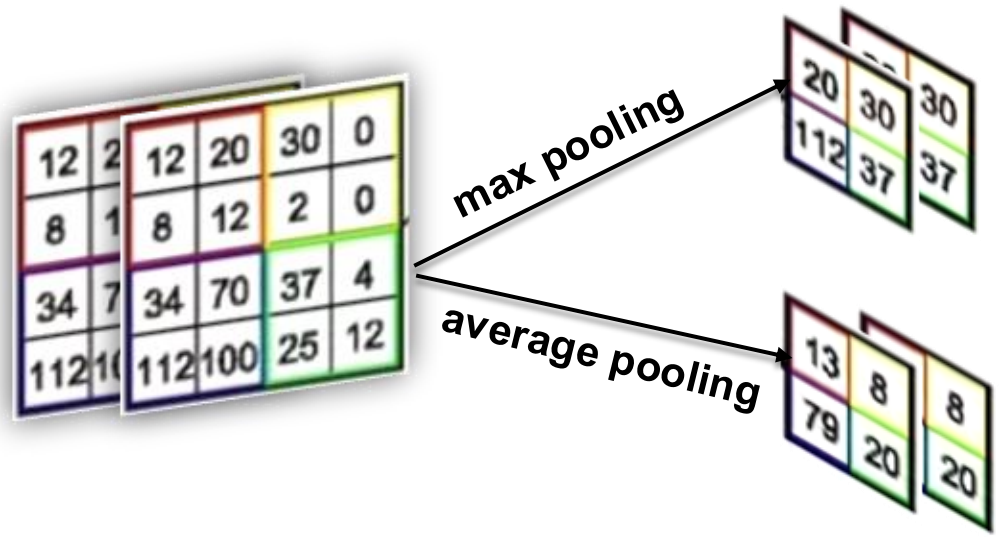


- CNN cannot capture **spatial relationships** among objects
  - How **spatially related** of two eyes, nose and eye, eye and mouth

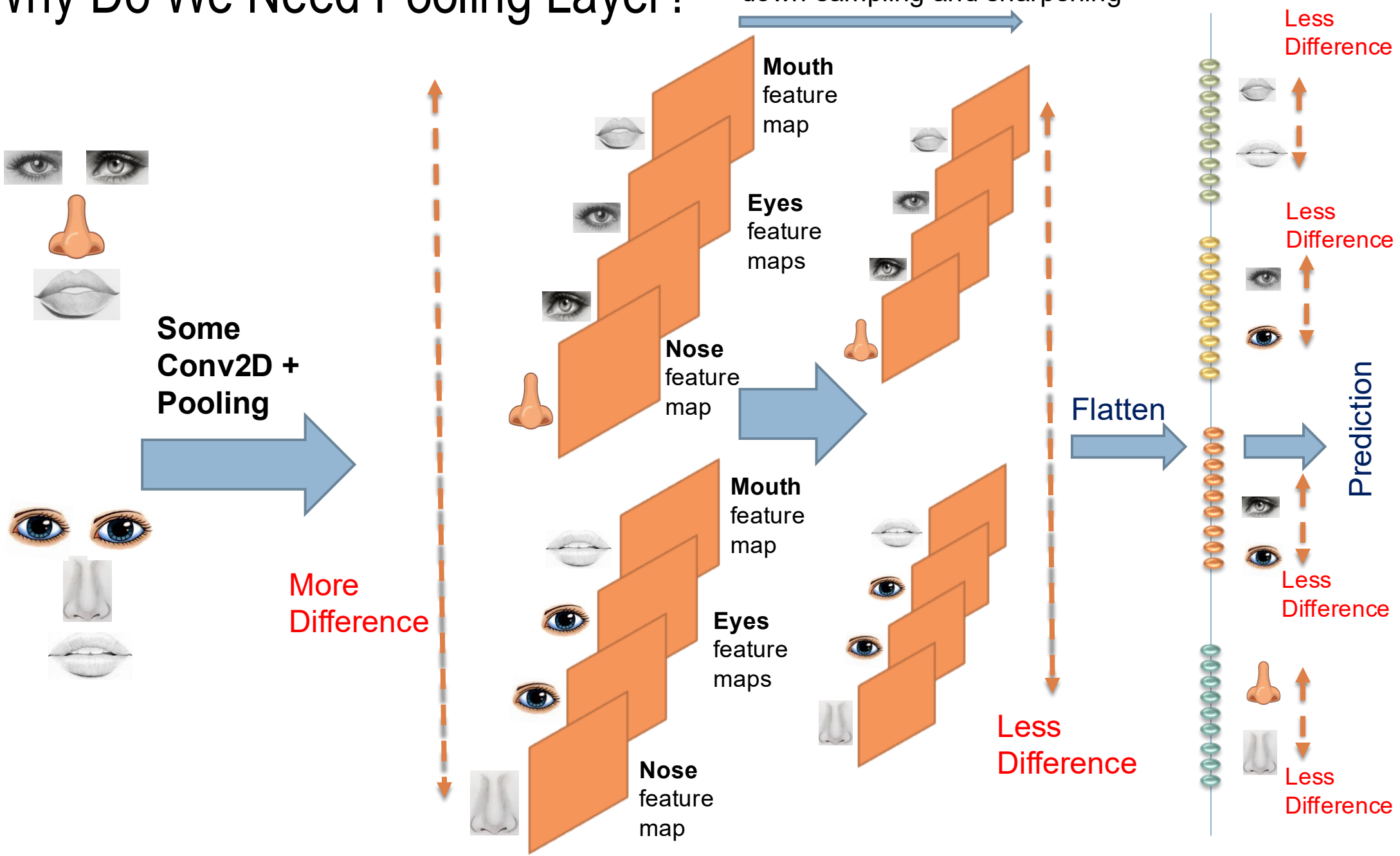


Pooling layer

- Max/average-pooling is **locally invariant** and applied **independently** to **each feature map**
- Reduce the size of feature map to be more manageable
  - Smaller size for flattening
- Reduce the difference of output tensors



Why Do We Need Pooling Layer?



Some comments from Hinton

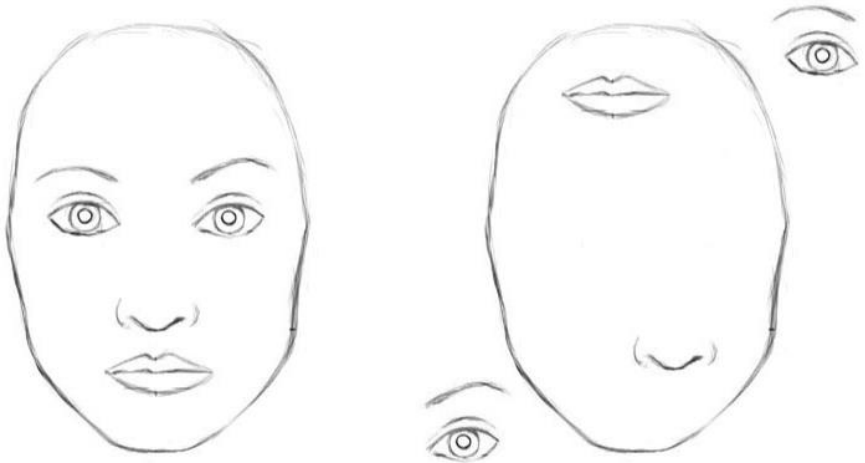
Dynamic Routing Between Capsules

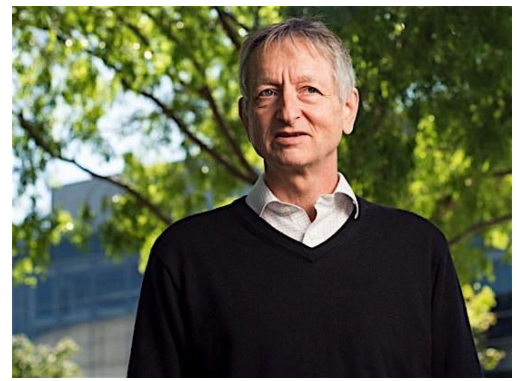
Sara Sabour **Nicholas Frosst**

Geoffrey E. Hinton
Google Brain
Toronto
{sasabour, frosst, geoffhinton}@google.com

Paper link: https://arxiv.org/pdf/1710.09829.pdf

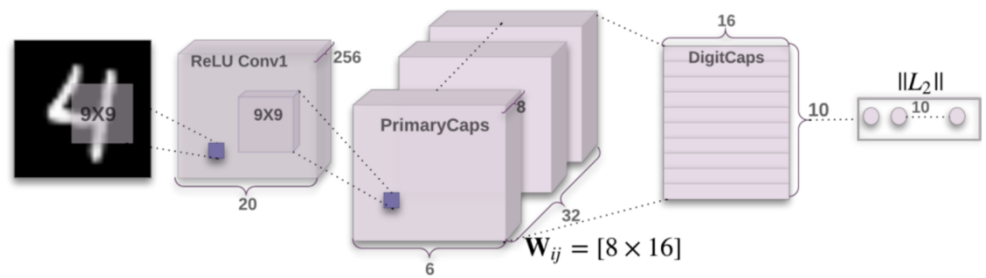
(Source: medium.com)





Hinton: "The pooling operation used in convolutional neural networks is a big mistake and the fact that it works so well is a disaster."

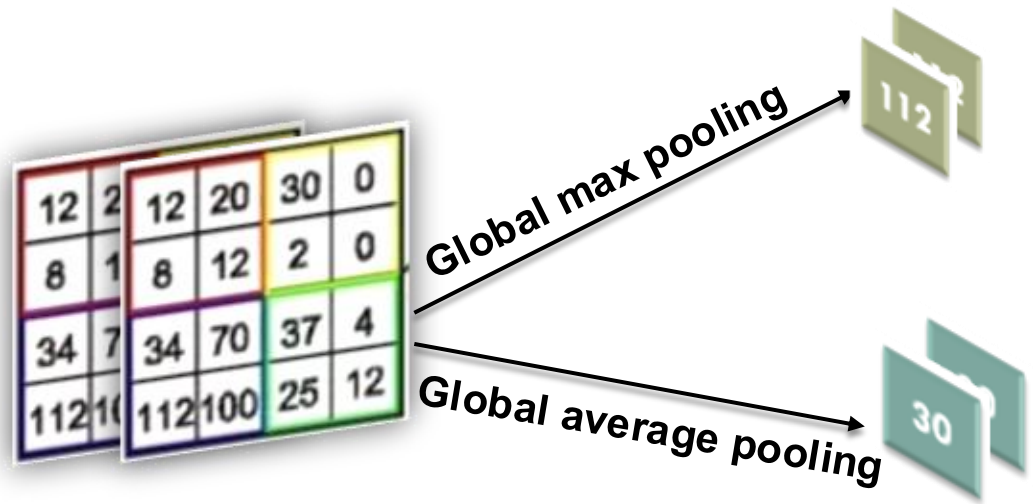
- CNNs cannot capture the **spatial relationships** among objects, hence it **wrongly predicts every face** with **two eyes**, **one nose**, and **one mouth** as a **human face**.



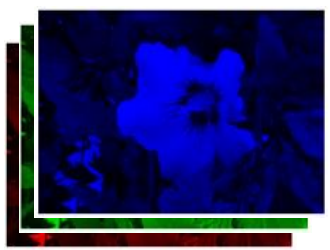
(Source: original paper of **Capsule Net**)

Global pooling layer

- Like **pooling layer**, but the **kernel size** is set to **input size**
  - **Each feature map** becomes a **single neuron**
  - **Global average pooling** (GAP)
  - **Global max pooling** (GMP)
  - **4D input tensor** [batch_size, in_channel, in_height, in_width]→ **2D output tensor** [batch_size, in_channel]



Global Pooling Layer



[3,32,32]



Input Layer

Filters [4,3,4,4]

Filter 1

[3,4,4]

Filter 2

[3,4,4]

Filter 3

[3,4,4]

Filter 4

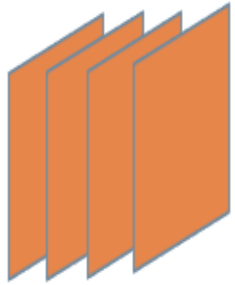
[3,4,4]

Conv2D 1

padding= 0
strides = (2,2)

Conv2d(3,4,kernel_size=4, stride=2, padding=0)

Feature volume
Feature maps



[4,15,15]

Latent representation

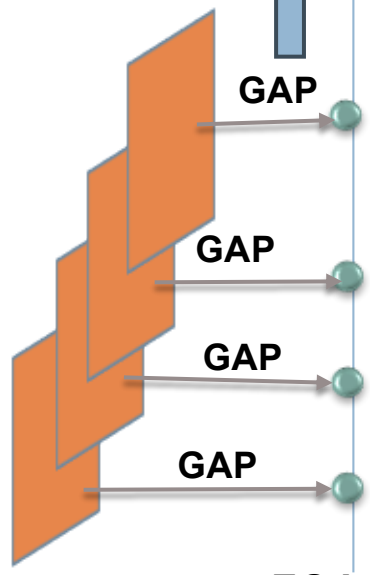
Pooling layer

pool-size=(2,2)
strides= (2,2)
padding= 1

MaxPool2d(kernel_size=2, stride=2, padding=1)

```
AdaptiveAvgPool2d((1, 1))
Flatten(1)
```

Feature volume
Feature maps



[4,8,8]

Latent representation

FC layer

4 neurons

softmax



Output layer

10 neurons
for 10 classes

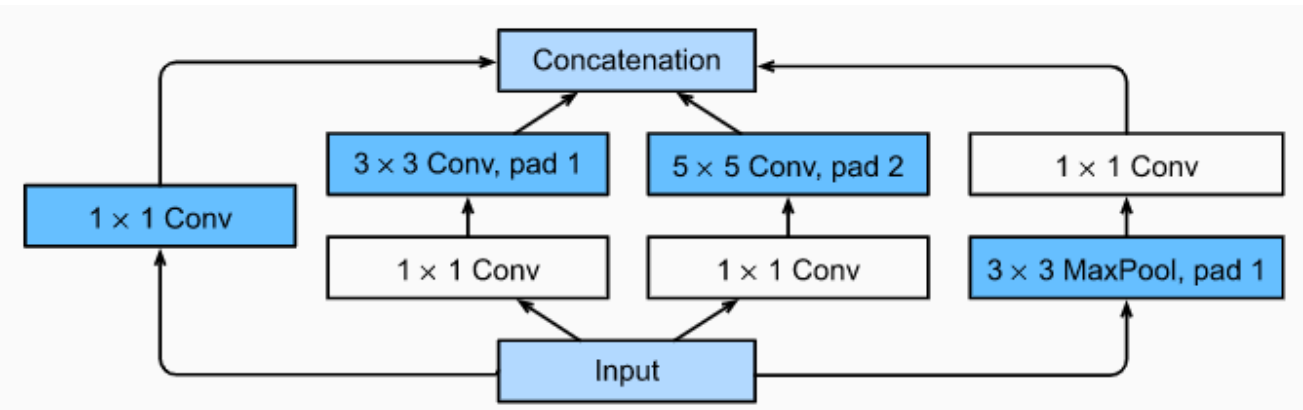


SOTA CNNs

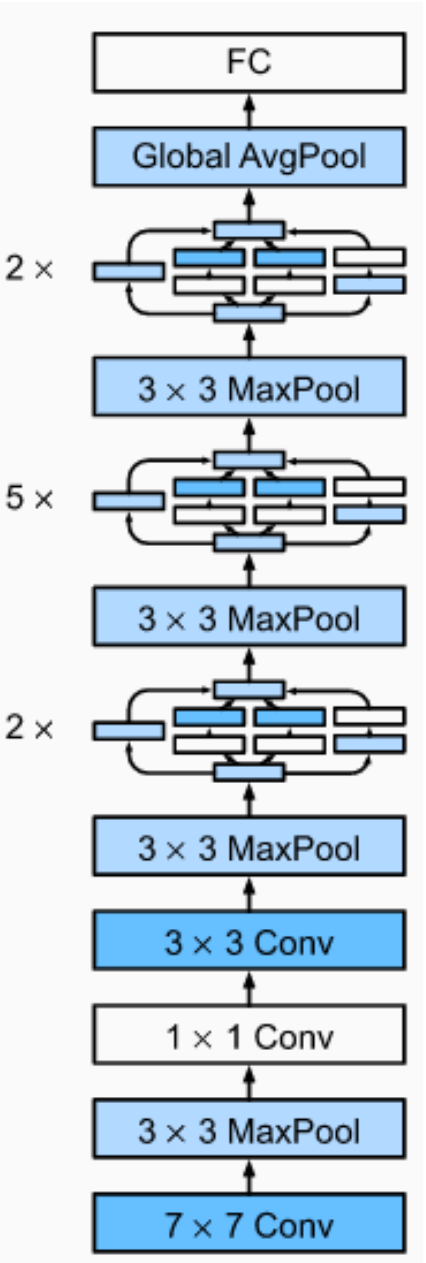
GoogleNet

Szegedy et al. '14

- **Won ImageNet** competition **in 2014**.
- Address the question of **which sized** convolution kernels are **best**.
  - Previous popular networks employed choices as **small as 1×1** and as **large as 7×7**.
  - Sometimes it can be **advantageous** to employ a **combination** of **variously-sized kernels**.
- The building-block is **Inception block**



Inception block



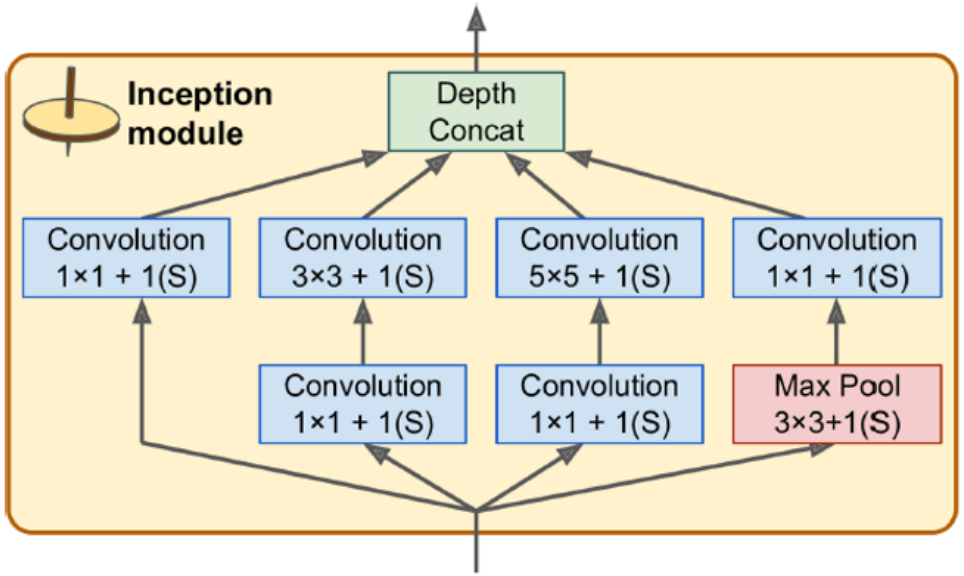
(Source:Dive into DL)

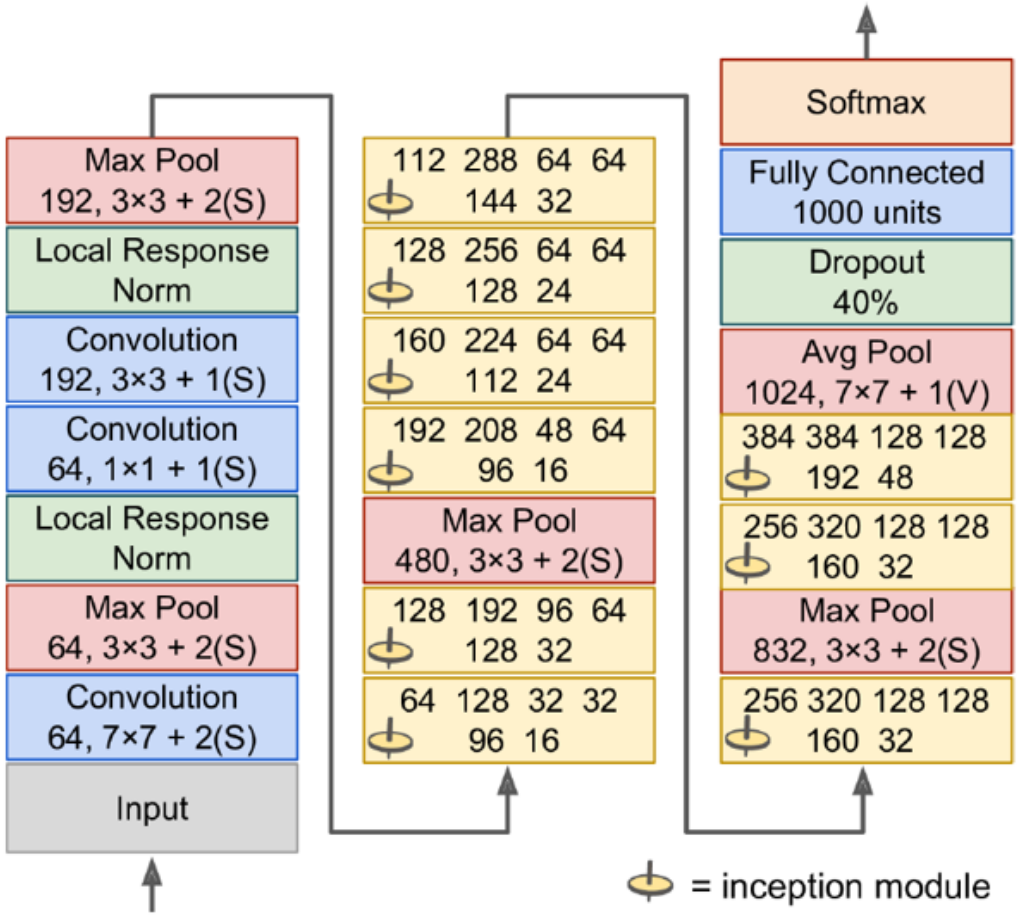
MONASH University

GoogleNet

Szegedy et al. ‘14

22 layers

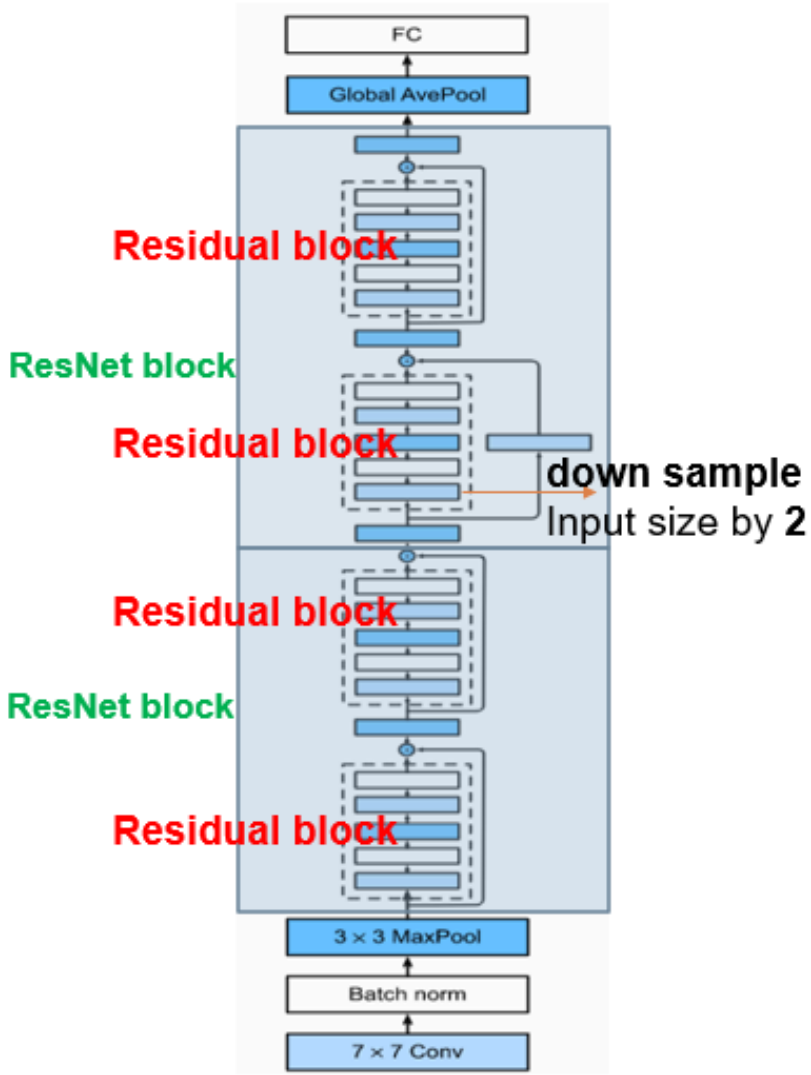




(Source: Hands On, Ch. 15)

ResNet

- **Won ImageNet** competition **in 2015**
- Many ResNet blocks
- Each ResNet block contains **many residual blocks**
  - Number of residual blocks
  - Number of channels for residual blocks
- Each ResNet block
  - From the **second block**, the first **residual block**
    - Use **1x1 Conv skip connection**
  - Otherwise
    - Use **standard skip connection**

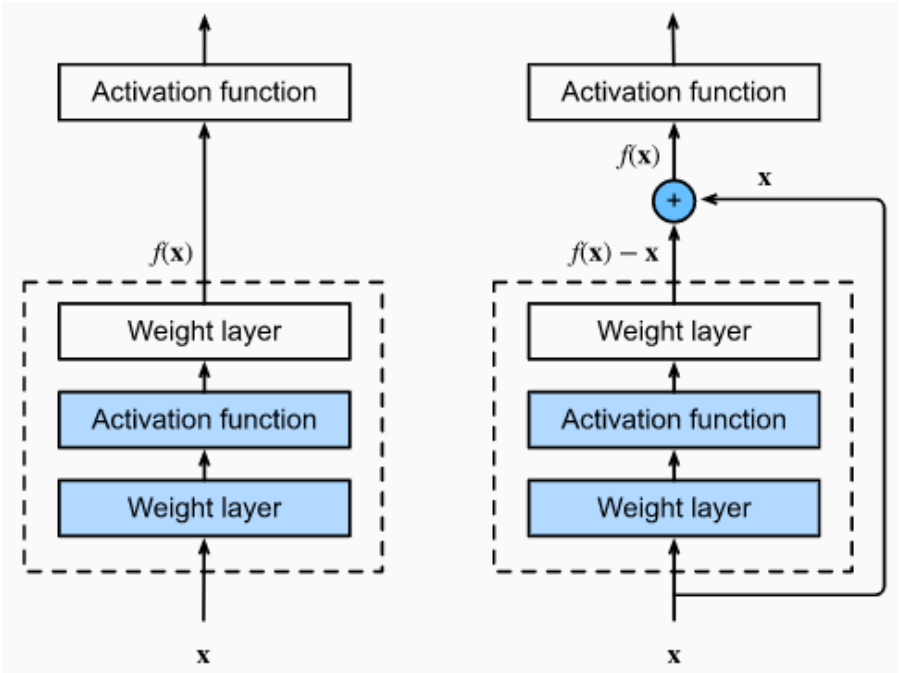


ResNet architecture

ResNet

Residual Block

(Source:Dive into DL)

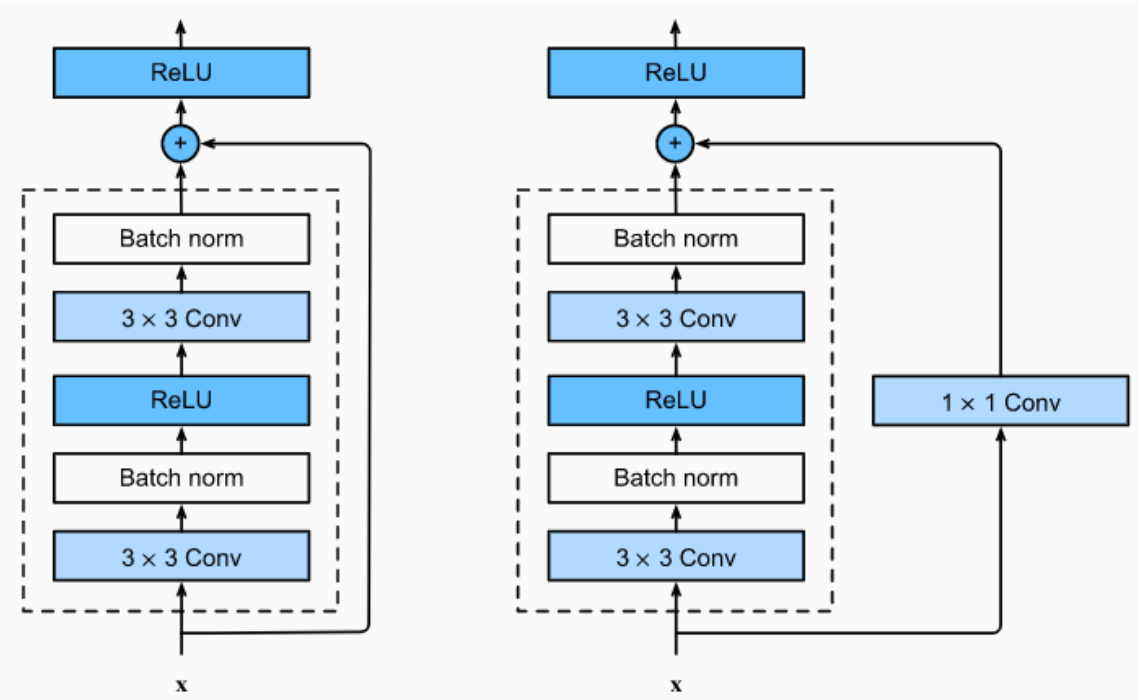


Without skip connection **With skip connection**

- Learn $f(x) = g(x) + x$ where $g(x) = f(x) - x$
  - The **model expressiveness** is the **same** as previous
  - The **gradient** $\nabla f(x) = \nabla g(x) + \mathbf{1}$ **looks better** where $\mathbf{1}$ is the vector of **all** 1 with the same shape as x

ResNet

Residual Block



Not use 1x1 Conv **Use 1x1 Conv**

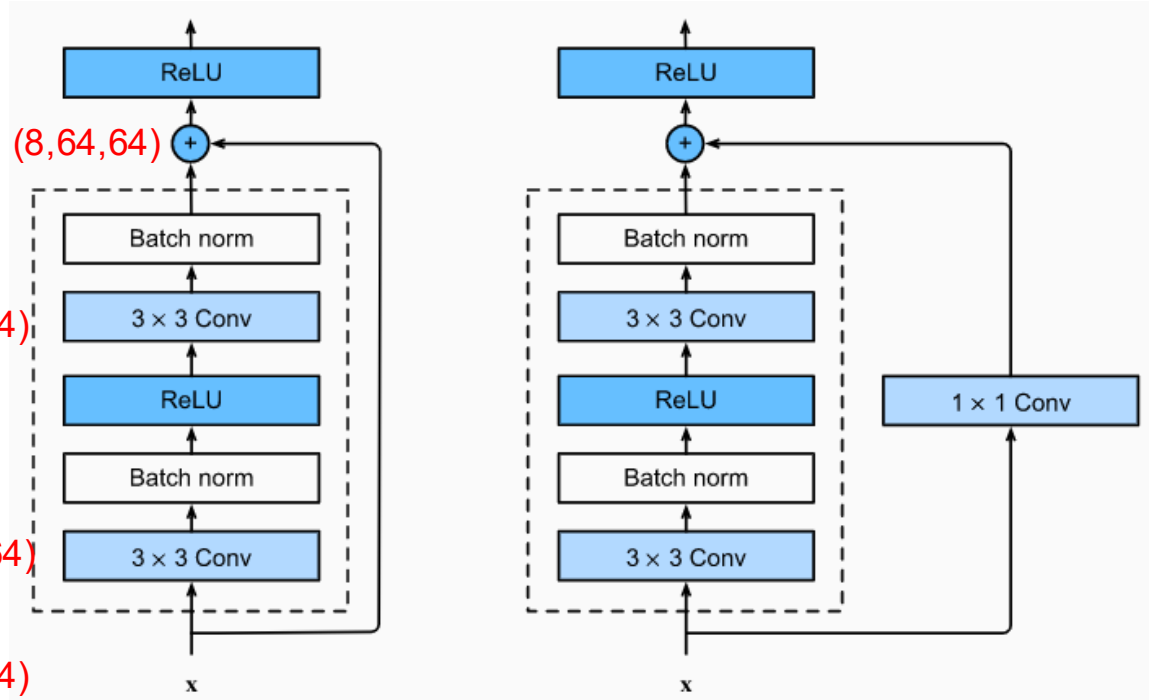
- ResNet follows VGG's full **3×3 convolutional layer** design.
  - The residual block has two **3×3 convolutional layers** with the **same number of output channels**.
  - Each **convolutional layer** is followed by a **batch normalization layer** and a **ReLU activation function**.
  - **Skip these two convolution operations** and **add the input directly** before the final ReLU activation function.
  - Requires that the output of **the two convolutional layers** must be of **the same shape** as the input, so that they can be added together.
    - If we want to change the number of channels, we need to introduce an additional **1×1 convolutional layer** to transform the input into the **desired shape** for the addition operation.

ResNet

Residual Block

padding=1
strides= (1,1)
#filters=8

padding=1
strides= (1,1)
#filters=8

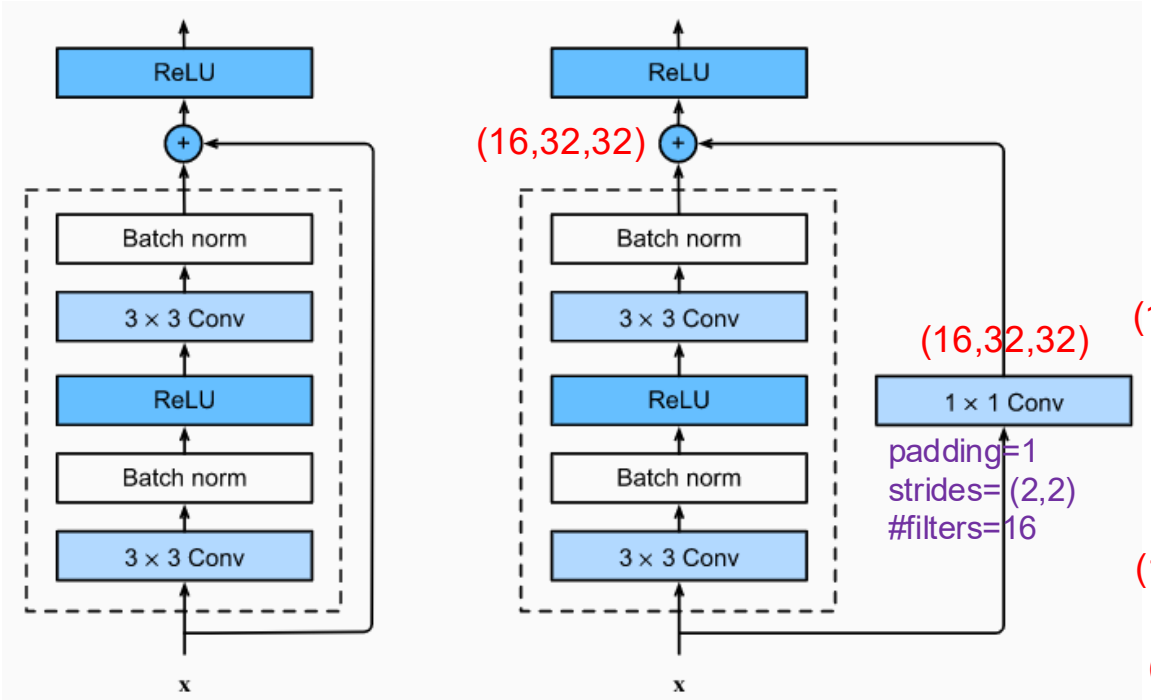


Not use 1x1 Conv **Use 1x1 Conv**

- ResNet follows VGG's full **3×3 convolutional layer** design.
  - The residual block has two **3×3 convolutional layers** with the **same number of output channels**.
  - Each **convolutional layer** is followed by a **batch normalization layer** and a **ReLU activation function**.
  - **Skip these two convolution operations** and **add the input directly** before the final ReLU activation function.
  - Requires that the output of **the two convolutional layers** must be of **the same shape** as the input, so that they can be added together.
    - If we want to change the number of channels, we need to introduce an additional **1×1 convolutional layer** to transform the input into the **desired shape** for the addition operation.

ResNet

Residual Block



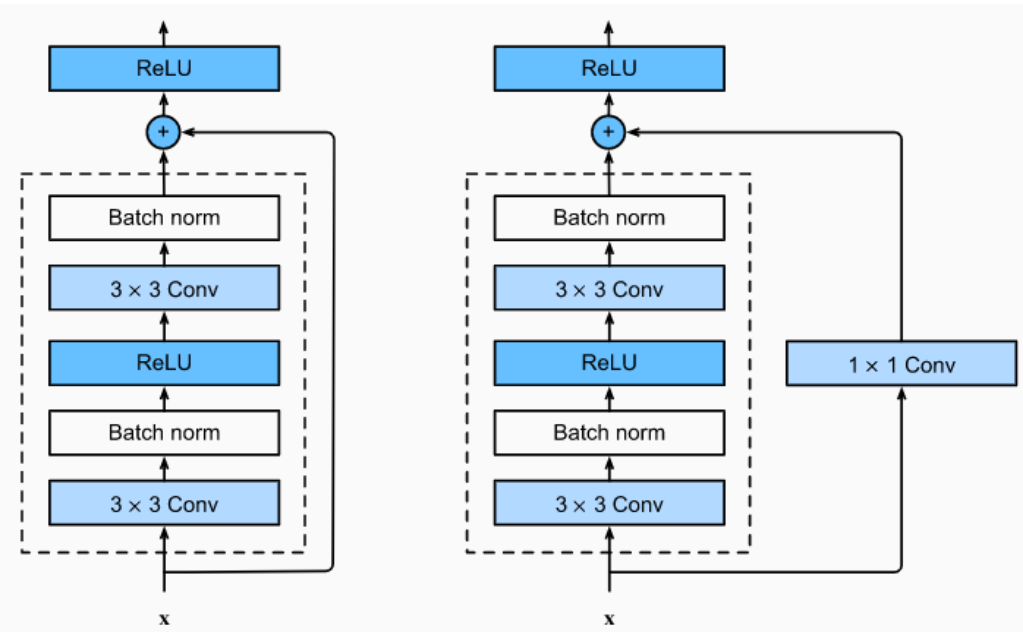
Not use 1x1 Conv **Use 1x1 Conv**

- ResNet follows VGG's full **3×3 convolutional layer** design.
  - The residual block has two **3×3 convolutional layers** with the **same number of output channels**.
  - Each **convolutional layer** is followed by a **batch normalization layer** and a **ReLU activation function**.
  - **Skip these two convolution operations** and **add the input directly** before the final ReLU activation function.
  - Requires that the output of **the two convolutional layers** must be of **the same shape** as the input, so that they can be added together.
    - If we want to change the number of channels, we need to introduce an additional **1×1 convolutional layer** to transform the input into the **desired shape** for the addition operation.

Implementation of Residual Block

```python
class Residual(nn.Module):
    def __init__(self, num_channels, use_1x1conv=False, strides=1):
        super().__init__()
        self.conv1 = nn.LazyConv2d(num_channels, kernel_size=3, stride=strides, padding=1)
        self.conv2 = nn.LazyConv2d(num_channels, kernel_size=3, padding=1)
        self.conv3 = None
        if use_1x1conv:
            self.conv3 = nn.LazyConv2d(num_channels, kernel_size=1, stride=strides)
        self.bn1 = nn.BatchNorm2d(num_channels)
        self.bn2 = nn.BatchNorm2d(num_channels)
        self.relu = nn.ReLU()

    def forward(self, X):
      Y = self.relu(self.bn1(self.conv1(X)))
      Y = self.bn2(self.conv2(Y))
      if self.conv3 is not None:
        X = self.conv3(X)
      Y += X
      return self.relu(Y)
```




```python
blk = Residual(3)
X = torch.rand((4, 3, 6, 6))
Y = blk(X)
print(Y.shape)
```

```
torch.Size([4, 3, 6, 6])
```

```python
blk = Residual(num_channels=3, use_1x1conv=True, strides=1)
X = torch.rand((10, 3, 32, 32))
Y = blk(X)
print(Y.shape)
```

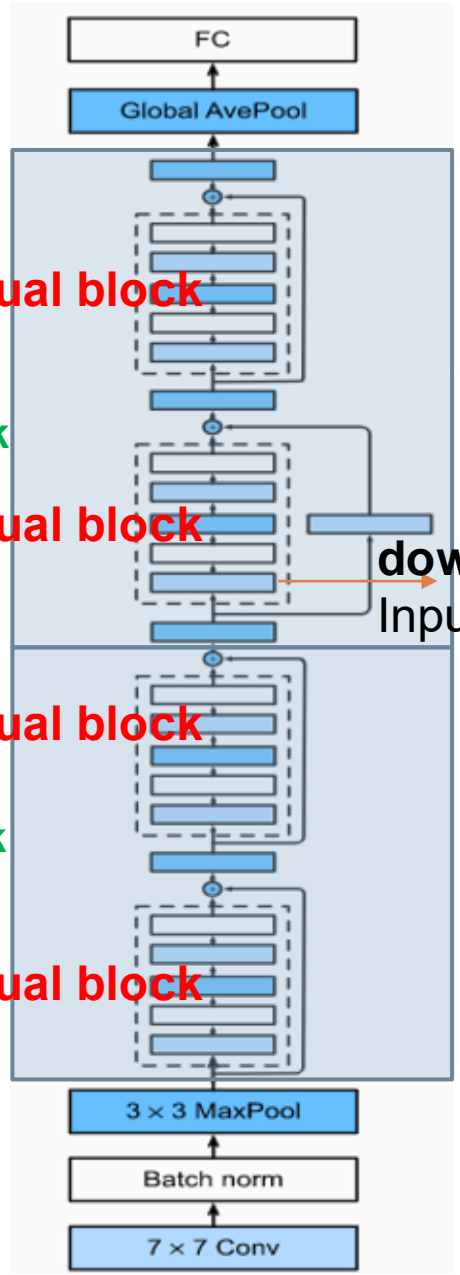
```
torch.Size([10, 3, 32, 32])
```

```python
blk = Residual(6, use_1x1conv=True, strides=2)
blk(X).shape
```

```
torch.Size([4, 6, 3, 3])
```

(Source:Dive into DL)

Implementation of ResNet



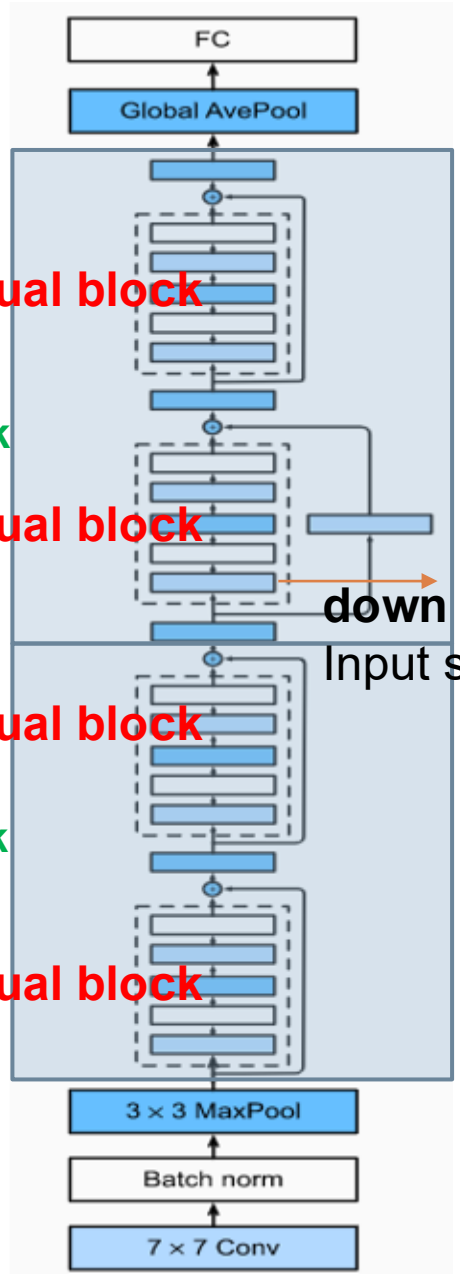
(Source:Dive into DL)

```
class ResnetBlock(nn.Module):
    def __init__(self, num_channels, num_residuals, first_block=False, **kwargs):
        super(ResnetBlock, self).__init__(**kwargs)
        self.residual_layers = []
        for i in range(num_residuals):
            if i == 0 and not first_block:
                self.residual_layers.append(
                    Residual(num_channels, use_1x1conv=True, strides=2))
            else:
                self.residual_layers.append(Residual(num_channels))
        self.residual_blk = nn.ModuleList(self.residual_layers)

    def forward(self, X):
        for layer in self.residual_blk:
            X = layer(X)
        return X
```

```
class create_ResNet(nn.Module):
  def __init__(self):
    super().__init__()
    self.layers = nn.ModuleList([
                  nn.LazyConv2d(64, kernel_size=7, stride=2, padding=3),
                  nn.LazyBatchNorm2d(),
                  nn.ReLU(),
                  nn.MaxPool2d(kernel_size=3, stride=2, padding=1),
                  ResnetBlock(64, 2, first_block=True),
                  ResnetBlock(128, 2),
                  ResnetBlock(256, 2),
                  nn.AdaptiveAvgPool2d((1, 1)),
                  nn.Flatten(1),
                  nn.LazyLinear(10),
                  # nn.Softmax(dim=-1)
                  ])
  def forward(self, X):
    for _, layer in enumerate(self.layers):
      X = layer(X)
    return X
```

Implementation of ResNet



(Source:Dive into DL)

```python
class ResnetBlock(nn.Module):
    def __init__(self, num_channels, num_residuals, first_block=False, **kwargs):
        super(ResnetBlock, self).__init__(**kwargs)
        self.residual_layers = []
        for i in range(num_residuals):
            if i == 0 and not first_block:
                self.residual_layers.append(
                    Residual(num_channels, use_1x1conv=True, strides=2))
            else:
                self.residual_layers.append(Residual(num_channels))
        self.residual_blk = nn.ModuleList(self.residual_layers)

    def forward(self, X):
        for layer in self.residual_blk:
            X = layer(X)
        return X
```

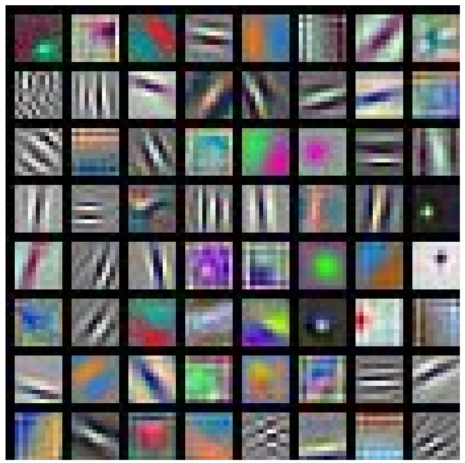
```python
class create_ResNet(nn.Module):
  def __init__(self):
    super().__init__()
    self.layers = nn.ModuleList([
(1, 64,112,112) ⇐ nn.LazyConv2d(64, kernel_size=7, stride=2, padding=3),
(1, 64,112,112) ⇐ nn.LazyBatchNorm2d(),
(1, 64,112,112) ⇐ nn.ReLU(),
 (1, 64,56,56) ⇐ nn.MaxPool2d(kernel_size=3, stride=2, padding=1),
 (1, 64,56,56) ⇐ ResnetBlock(64, 2, first_block=True),
(1, 128,28,28) ⇐ ResnetBlock(128, 2),
(1, 256,14,14) ⇐ ResnetBlock(256, 2),
  (1, 256,1,1) ⇐ nn.AdaptiveAvgPool2d((1, 1)),
    (1,256) ⇐ nn.Flatten(1),
    (1,10) ⇐ nn.LazyLinear(10),
                # nn.Softmax(dim=-1)
                ])
  def forward(self, X):
    for _, layer in enumerate(self.layers):
      X = layer(X)
    return X
```


Q&A
WHO?
WHERE?
WHEN?
WHAT?
How?
Why?
When?
Where?
HOW?

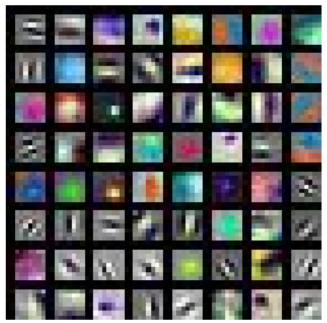
Visualizing CNNs: *Activation Map Visualization and GradCam*

First layer visualization

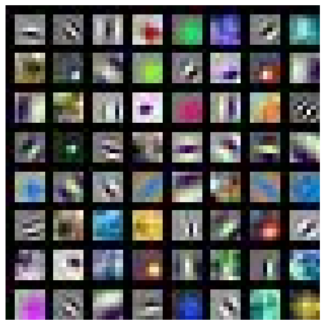
First layer visualization



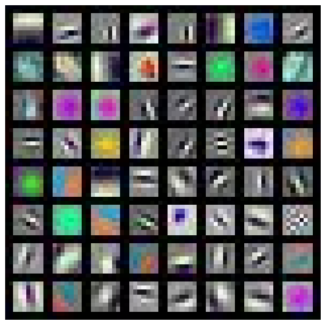
AlexNet:
64 x 3 x 11 x 11



ResNet-18:
64 x 3 x 7 x 7

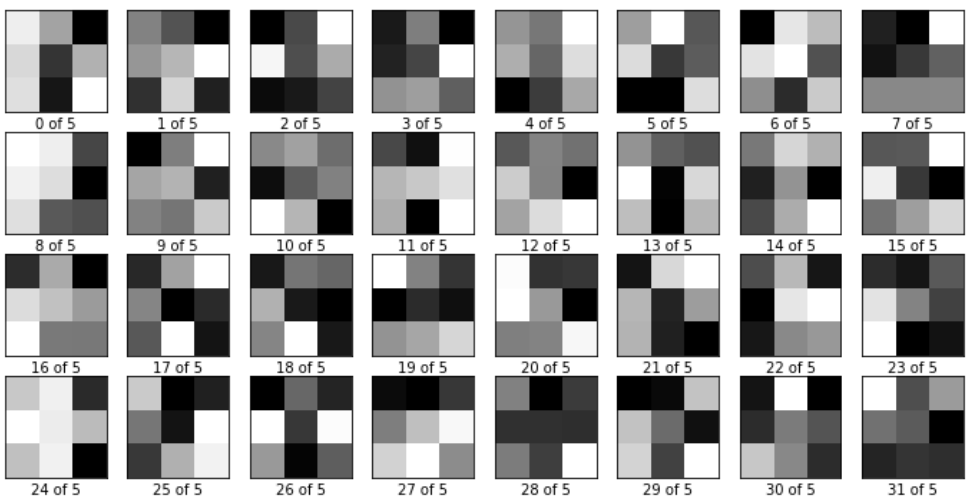


ResNet-101:
64 x 3 x 7 x 7

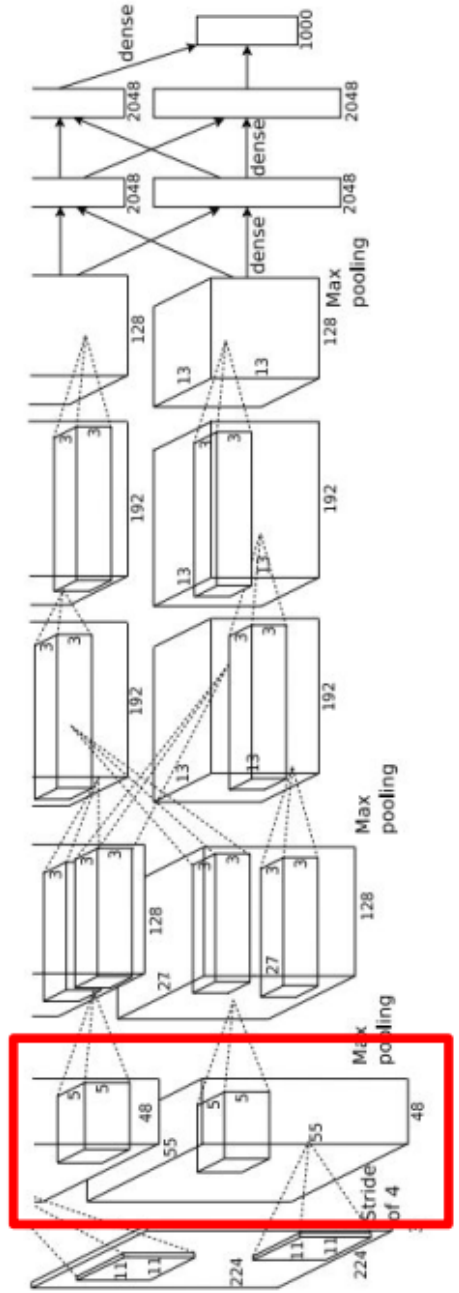


DenseNet-121:
64 x 3 x 7 x 7

Visualization of filters at higher layers is not that interesting



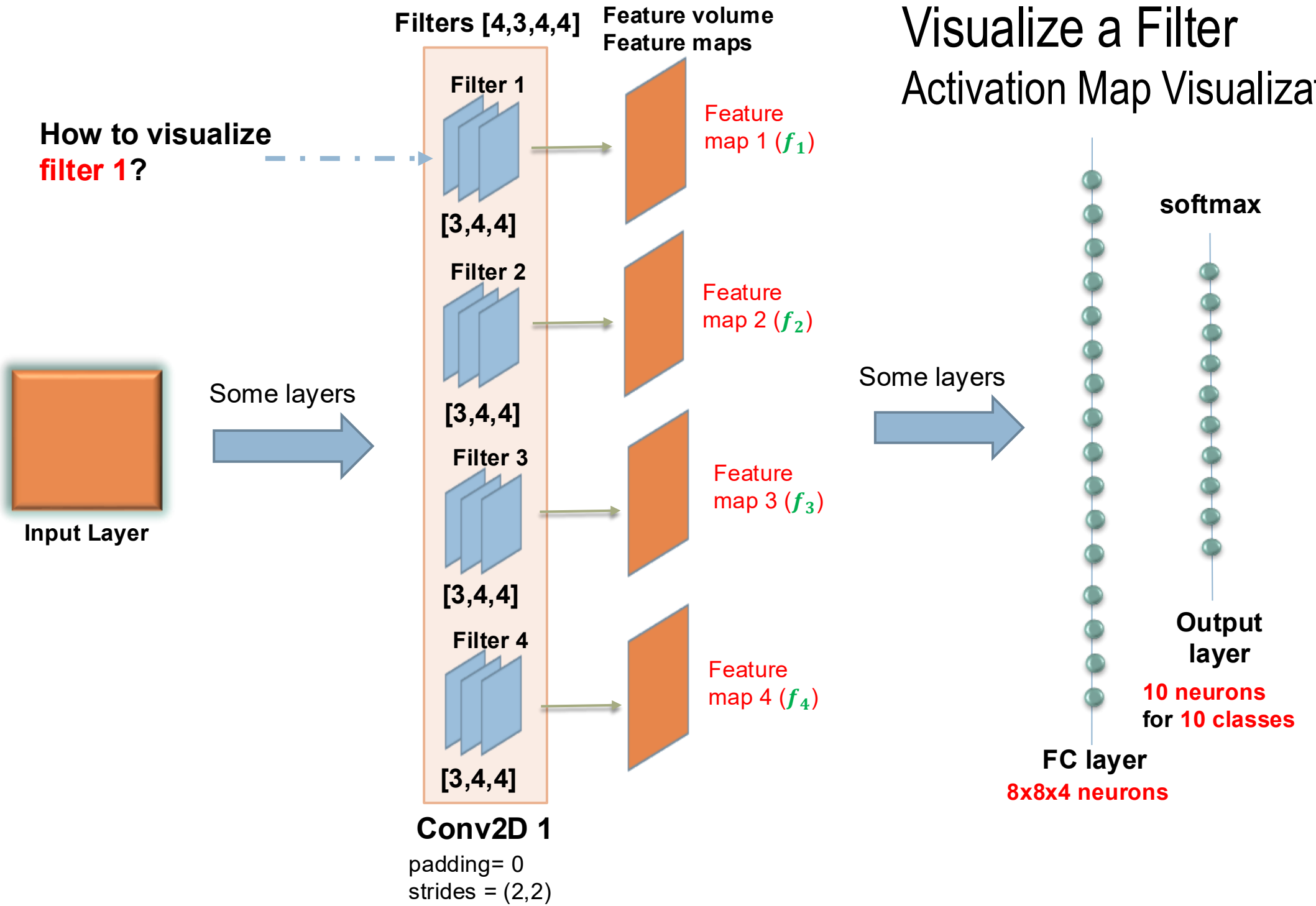
Tiny **3x3** filters



(CS231n: Convolutional Neural Networks for Visual Recognition", Stanford University, 2016)

Visualize a Filter

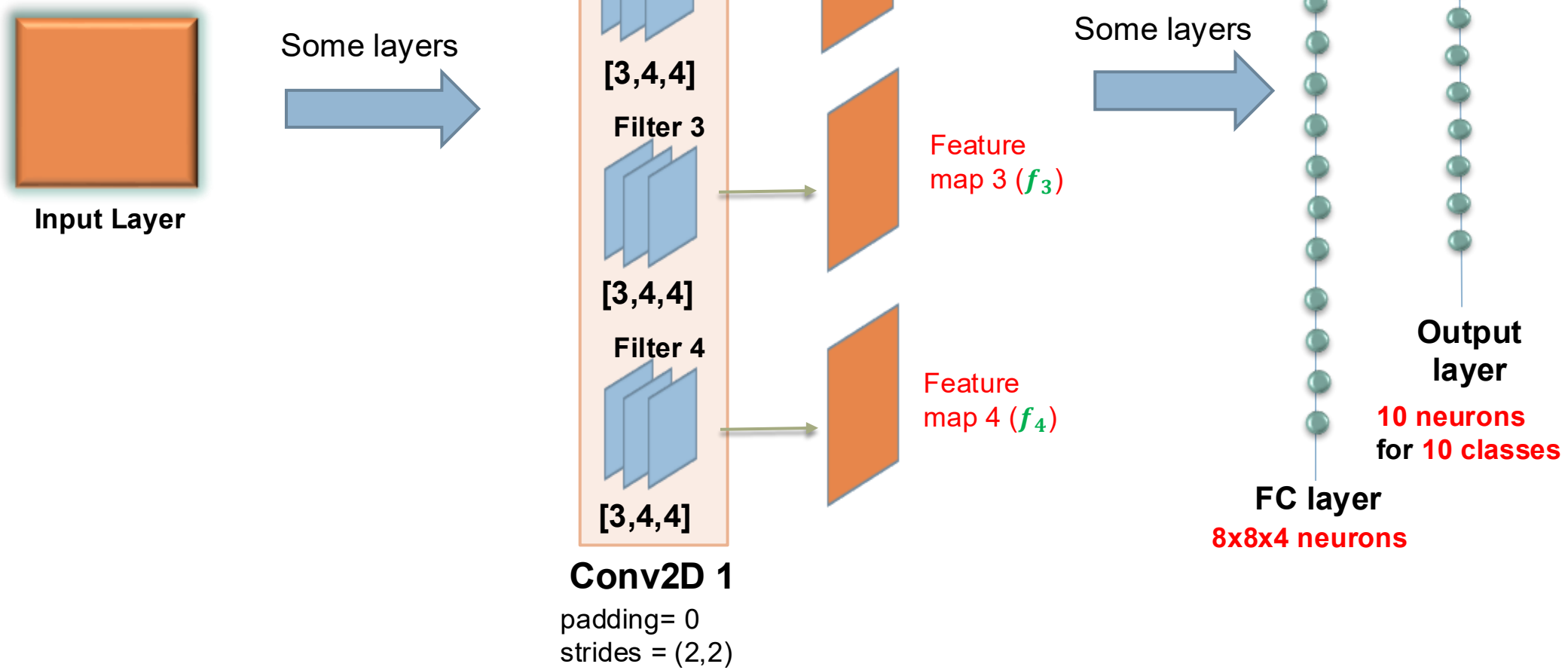
Activation Map Visualization



Visualize a Filter

Activation Map Visualization

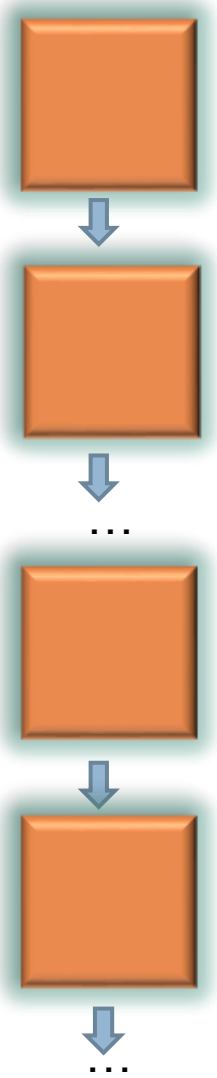
- How to visualize **filter 1**?
- Find input I^* that **maximally fires filter 1**.
- $I^* = argmax_I \, mean(f_1)$



Visualize a Filter

Activation Map Visualization

- How to visualize **filter 1**?
- Find input I^* that **maximally fires filter 1**.
- $I^* = argmax_I \, mean(f_1)$



I_0 initialized randomly

I_1

…

I_t

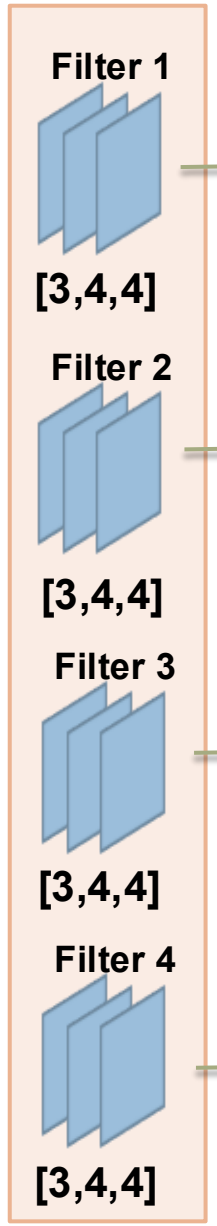
$I_{t+1} = I_t + \eta \nabla_I \mathcal{A}(I_t)$

$\mathcal{A}(I) = mean(f_1)$

…

Some layers





Conv2D 1

padding= 0

strides = (2,2)

Feature volume
Feature maps



Feature map 1 (f_1)

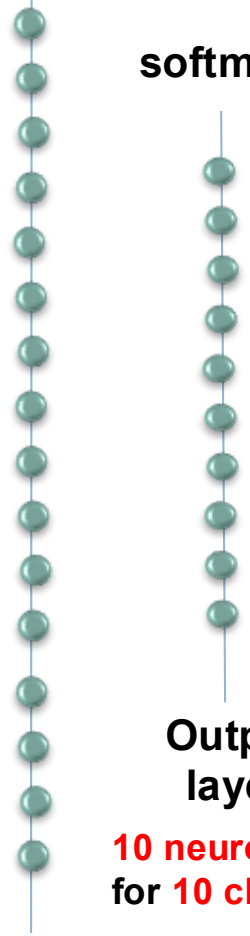
Feature map 2 (f_2)

Feature map 3 (f_3)

Feature map 4 (f_4)

Some layers





FC layer

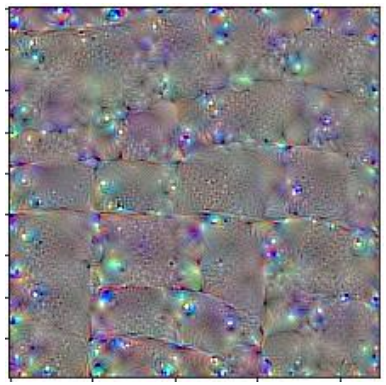
8x8x4 neurons

softmax

Output layer

10 neurons for 10 classes

Activation Map Visualization



Input image I

Feed to our CNN

Filter 1

1	1	1
1	1	1
1	1	1

Feature map 1 (f_1)

-1	23	21
1	26	17
-1	8	13

Filter 2

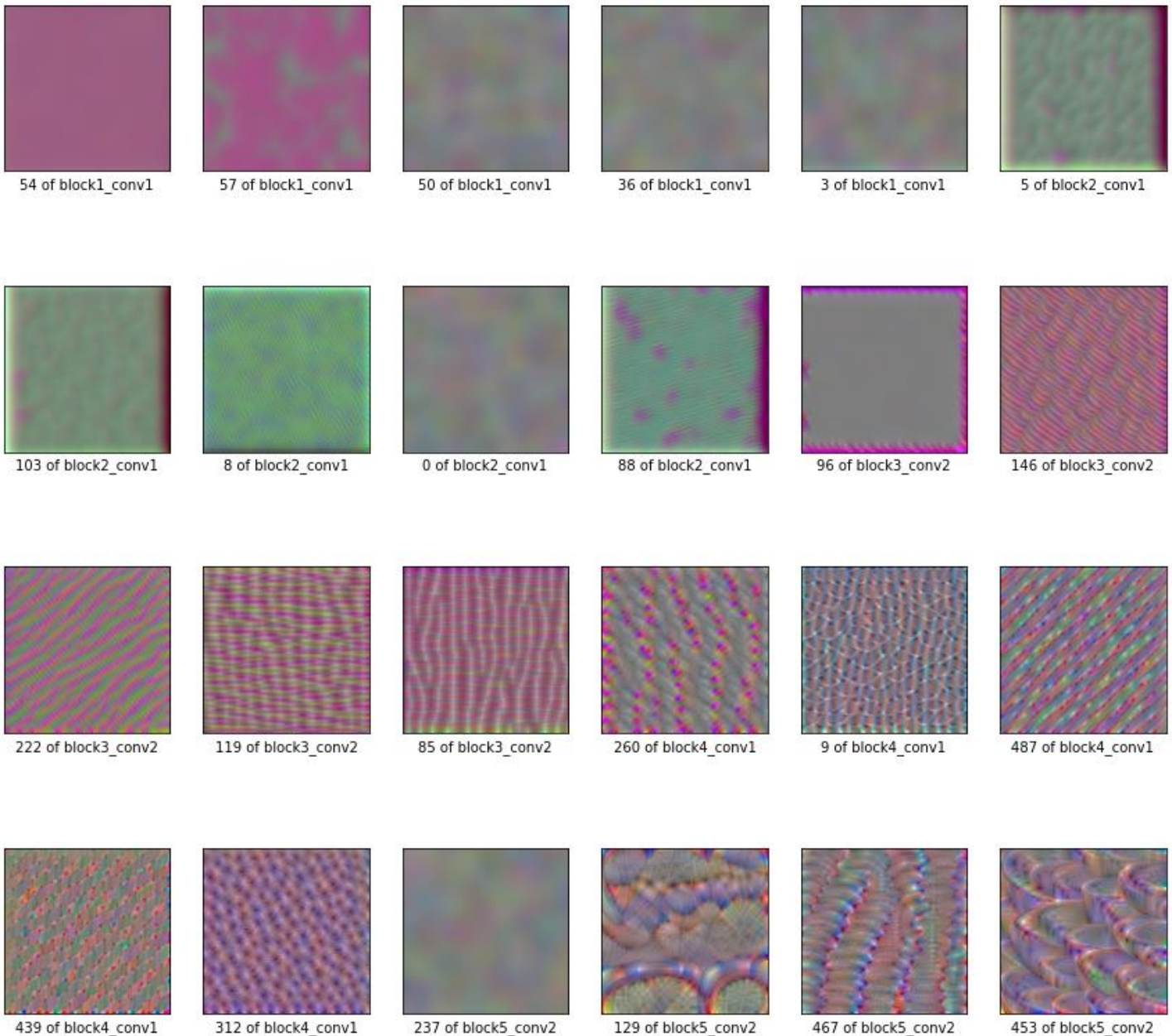
-1	-1	-1
-1	-1	-1
-1	-1	-1

Feature map 2 (f_2)

1	-23	-21
-1	-26	-17
1	-8	-13

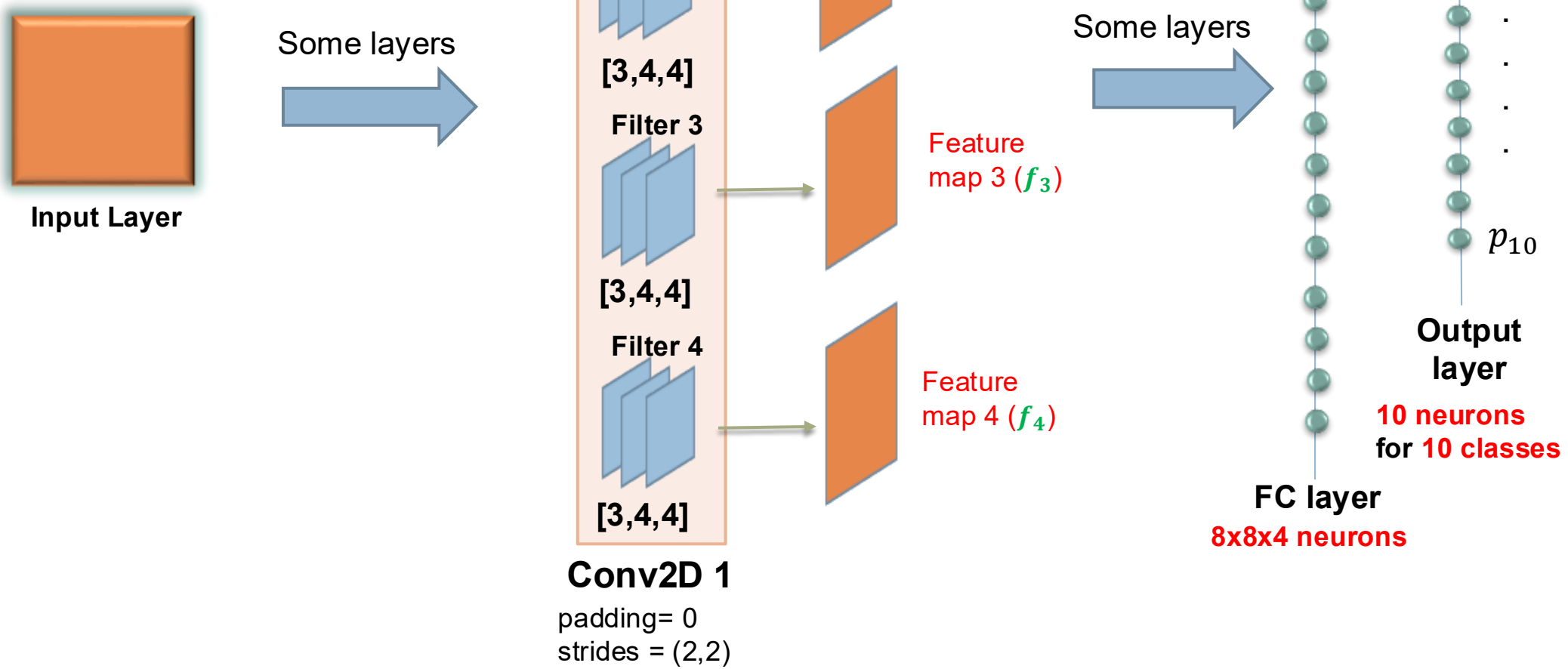
- Find **the input image I^*** that **maximally activate** a feature map **corresponding** to a filter
  - I^* shows the patterns in input images that can fire (maximally activate) a certain filter
- To visualize **filter 1,** we find the input I^* that **maximizes** average of feature map f_1
  - $I^* = argmax_I \mathcal{A}(I)$ where $\mathcal{A}(I) := mean(f_1) - \alpha\Omega(I)$
  - $\Omega(I)$ is a **norm** over I to encourage the sparsity
- Using **gradient ascent** for updating
  - $I_{t+1} = I_t + \eta \nabla_I \mathcal{A}(I_t)$
  - $\eta > 0$ is the learning rate

Activation map visualization of VGG19



Visualize a Class

- How to visualize **class 1**?
- Find input I^* that **maximizes prediction probability** p_1.
- $I^* = argmax_I\, p_1$



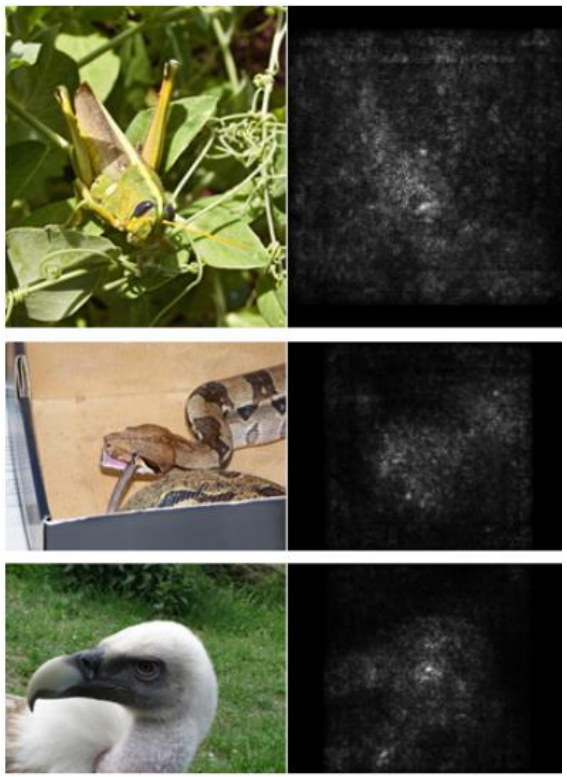
GradCam Model Explanability

- Which **part of image** that a CNN (e.g., ResNet 50) **highly bases on** to **make prediction**?



Original image | Gradcam | Guided Gradcam

GradCam Model Explainability



Salience/Heat map

Deep Inside Convolutional Networks: Visualising Image Classification Models and Saliency Maps

Karen Simonyan Andrea Vedaldi Andrew Zisserman
Visual Geometry Group, University of Oxford
{karen,vedaldi,az}@robots.ox.ac.uk

Link: https://arxiv.org/pdf/1312.6034



Which **part** of images **mostly contribute** to the **model prediction**?

$p = [p_1, \dots, p_{\hat{y}}, \dots, p_{1000}]$

$\hat{y}$ is a **predicted label**

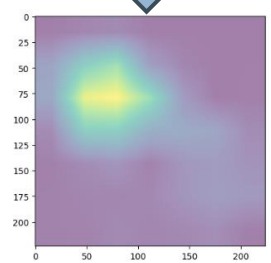
A layer with **3D tensor** feature maps [100,7,7]

$A = [A_1, A_2, \dots, A_{100}]$ with shape [100,7,7]

A_i with shape [7,7] is a feature map.

$heatmap = \sum_{i=1}^{100} sum\left(\frac{\partial p_{\hat{y}}}{\partial A_i}\right) \times A_i$ has shape [7,7]

Normalize *heatmap* and **resize** to [224, 224]



Salience/Heat map

Grad-CAM:
Visual Explanations from Deep Networks via Gradient-based Localization

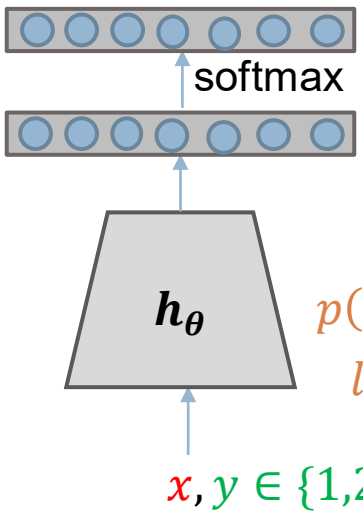
Ramprasaath R. Selvaraju[1*] Michael Cogswell[1] Abhishek Das[1] Ramakrishna Vedantam[1*]
Devi Parikh[1,2] Dhruv Batra[1,2]
[1]Georgia Institute of Technology [2]Facebook AI Research
{ramprs, cogswell, abhshkdz, vrama, parikh, dbatra}@gatech.edu

Link: https://arxiv.org/pdf/1610.02391



Adversarial Machine Learning

Some notions



$f(x;\theta) = softmax(h(x;\theta))$ (prediction probabilities)

$h(x;\theta)$ (logits)

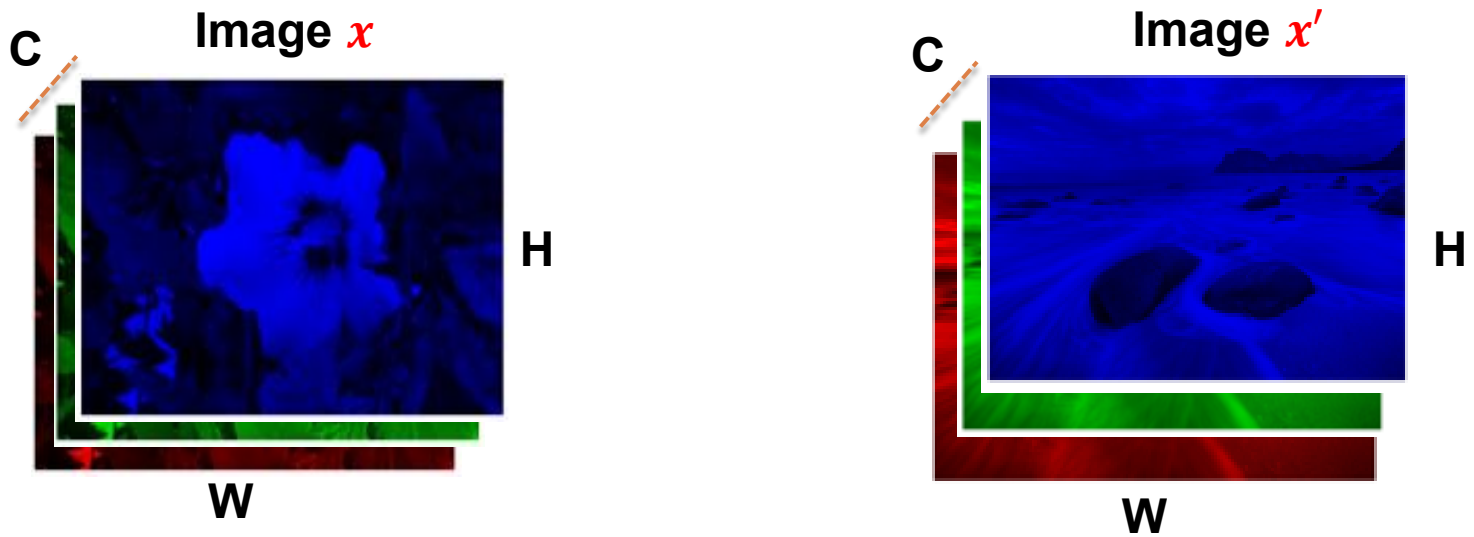
$$f(x;\theta) = [f_m(x;\theta)]_{m=1}^{M}$$

$$p(y = m \mid x) = f_m(x;\theta) \; for \; 1 \le m \le M$$

$$l(f(x;\theta), y) = CE\left(\mathbf{1}_y, f(x;\theta)\right) = -\log f_y(x;\theta)$$

$x, y \in \{1,2,\ldots,M\}$

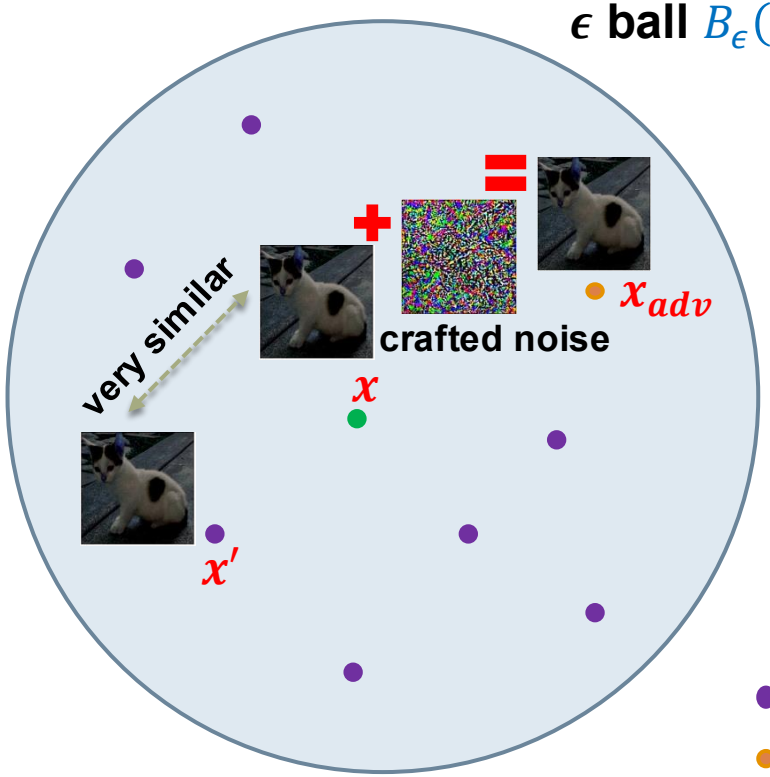
Distance between two images

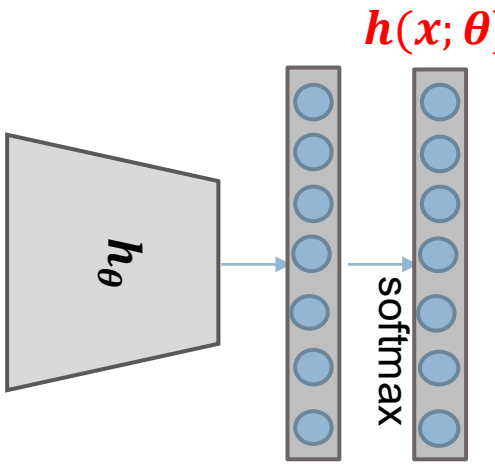


- x_{ijk} $(1 \leq i \leq C, 1 \leq j \leq H, 1 \leq k \leq W)$ is **the pixel** with coordinate $(height = j, width = k)$ on the channel (or slice) i of image x.
  - $0 \leq x_{ijk} \leq 255$ (original colour image) or $0 \leq x_{ijk} \leq 1$ (grey-scale image)
- L_2 distance
  - $\|x - x'\|_2 = \sqrt{\sum_{i=1}^{C} \sum_{j=1}^{H} \sum_{k=1}^{W} (x_{ijk} - x'_{ijk})^2}$
- L_1 distance
  - $\|x - x'\|_1 = \sum_{i=1}^{C} \sum_{j=1}^{H} \sum_{k=1}^{W} |x_{ijk} - x'_{ijk}|$
- L_∞ distance
  - $\|x - x'\|_\infty = \max_{1 \leq i \leq C, 1 \leq j \leq H, 1 \leq k \leq W} |x_{ijk} - x'_{ijk}|$

Adversarial examples

ϵ **ball** $B_\epsilon(x) = \{x' : \|x' - x\| \leq \epsilon\}$ with a **small** $\epsilon > 0$.





$$f(x;\theta) = softmax(h(x;\theta))$$

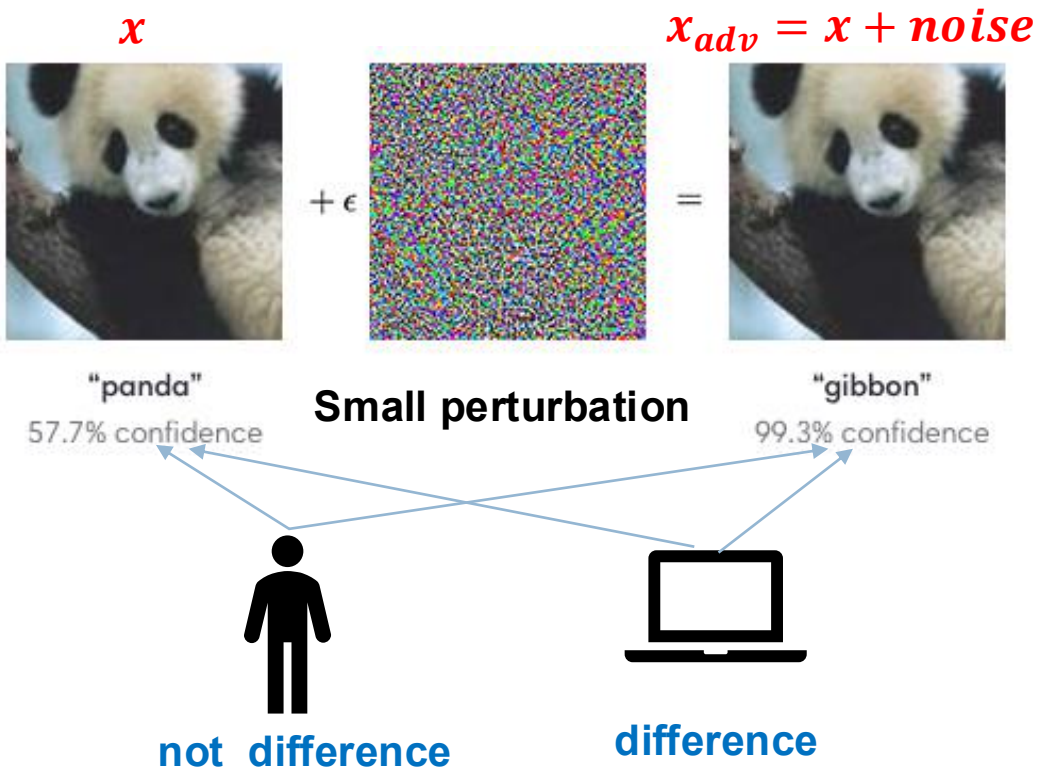
$$argmax_{1\leq i\leq M}\, f_i(x;\theta) = \hat{y} \neq \hat{y}_{adv} = argmax_{1\leq i\leq M}\, f_i(x_{adv};\theta)$$

- **Good** example which has **same predicted label** as x.
- **Adversarial** example which has **different predicted label** as x.

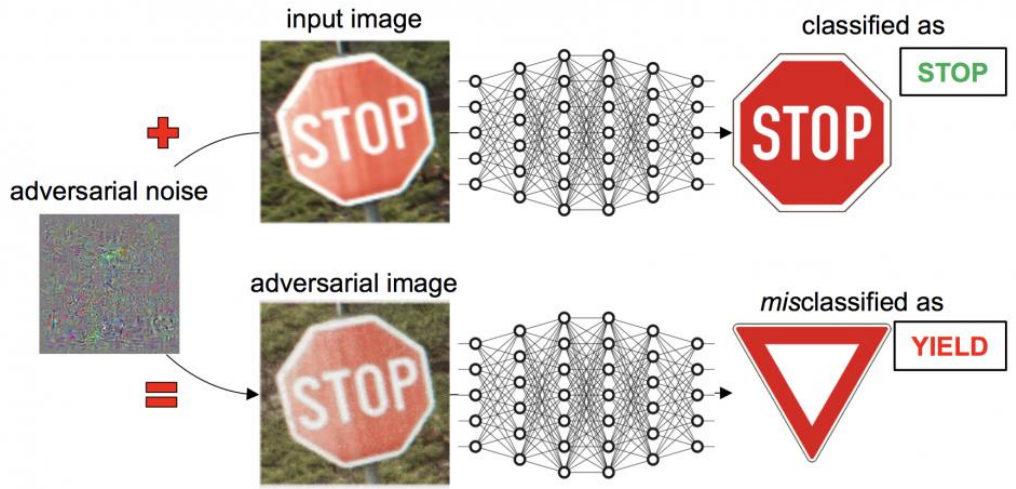
- The **true sense** of a **robust model**
  - If x' and x are **close**, the **predictions** of x' and x must be the **same**.
  - If $x' \in B_\epsilon(x)$ (i.e., $\|x' - x\| \leq \epsilon$), $argmax_{1\leq i\leq M}\, f_i(x;\theta) = \hat{y} = \hat{y}' = argmax_{1\leq i\leq M}\, f_i(x';\theta)$.
- Unfortunately, we can **easily** find **adversarial examples** $x_{adv} \in B_\epsilon(x)$ that can **fool** the classifier
  - $argmax_{1\leq i\leq M}\, f_i(x;\theta) = \hat{y} \neq \hat{y}_{adv} = argmax_{1\leq i\leq M}\, f_i(x_{adv};\theta)$

Adversarial examples

(Source: OpenAI)



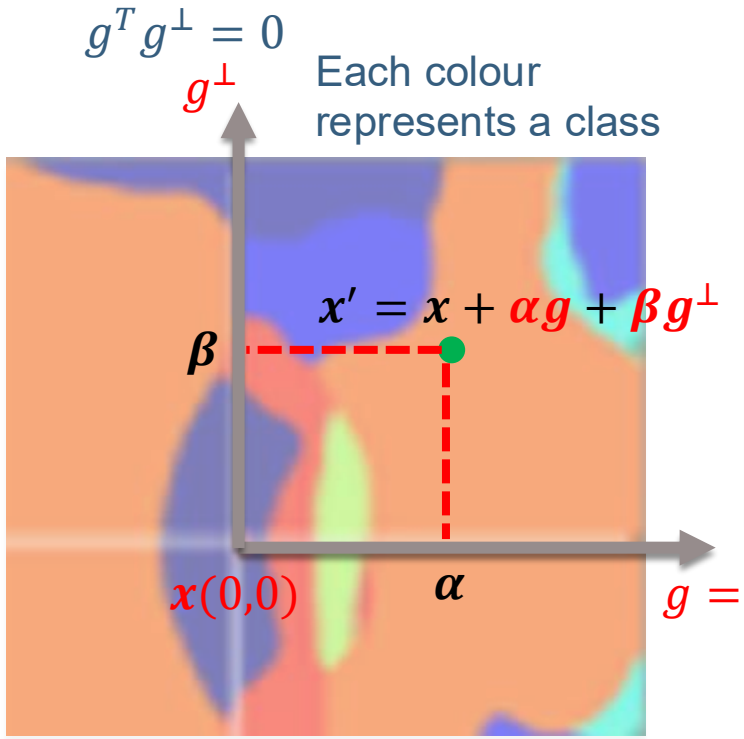
(Source: Internet)

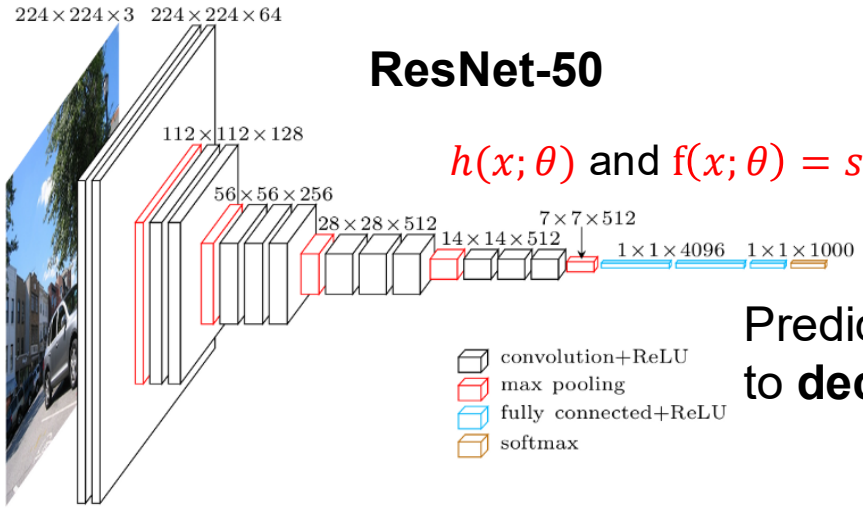


Misleading autonomous car driving system

- Adversarial example $\boldsymbol{x_{adv}}$ of a clean data example $\boldsymbol{x}$ w.r.t a model $f(.;\theta)$
  - $d(x_{adv}, x) = \|x_{adv} - x\| \leq \epsilon$ where $\|.\|$ is a norm (e.g., $\|.\|_1, \|.\|_2, \|.\|_\infty$)
  - f predicts x and x_{adv} with **different labels**, i.e., $argmax_i f_i(x;\theta) \neq argmax_i f_i(x_{adv};\theta)$

Visualization of decision regions





ResNet-50

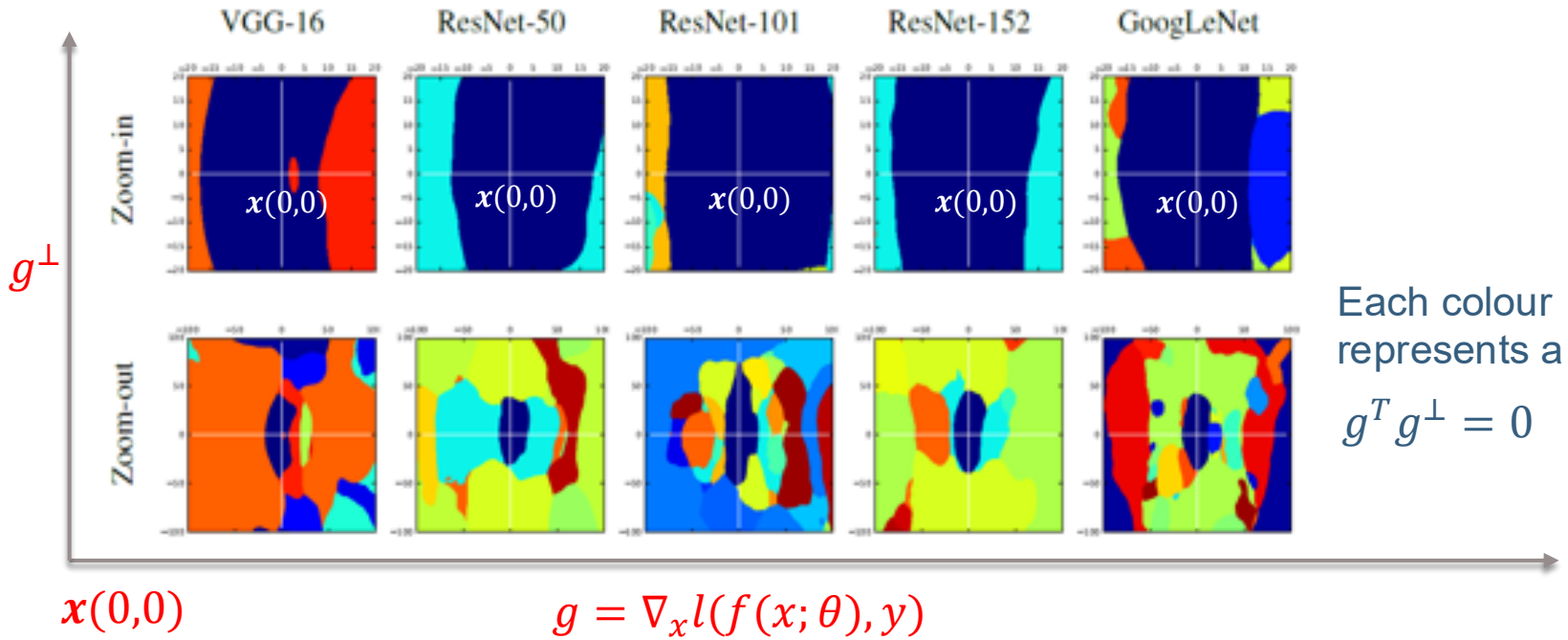
$h(x;\theta)$ and $f(x;\theta) = softmax(h(x;\theta))$

Predicts $x' = x + \alpha g + \beta g^\perp$ to **decide** the **colour**

Following the **gradient direction** is easy to **get out the blue region**.

- Start from $x = x + 0g + 0g^\perp$ (coordinate $(\mathbf{0}, \mathbf{0})$), if we **go along with** the **gradient direction** $g = \nabla_x l(f(x;\theta), y)$, we can reach $x_{adv} = x + \alpha g + 0g^\perp$ such that
  - x_{adv} is very close to x → human cannot tell how they **are different**→ the **same class** from human **perceptron**
  - x_{adv} is predicted to **different class** from x by the classifier
  - x_{adv} is called **adversarial example** of x

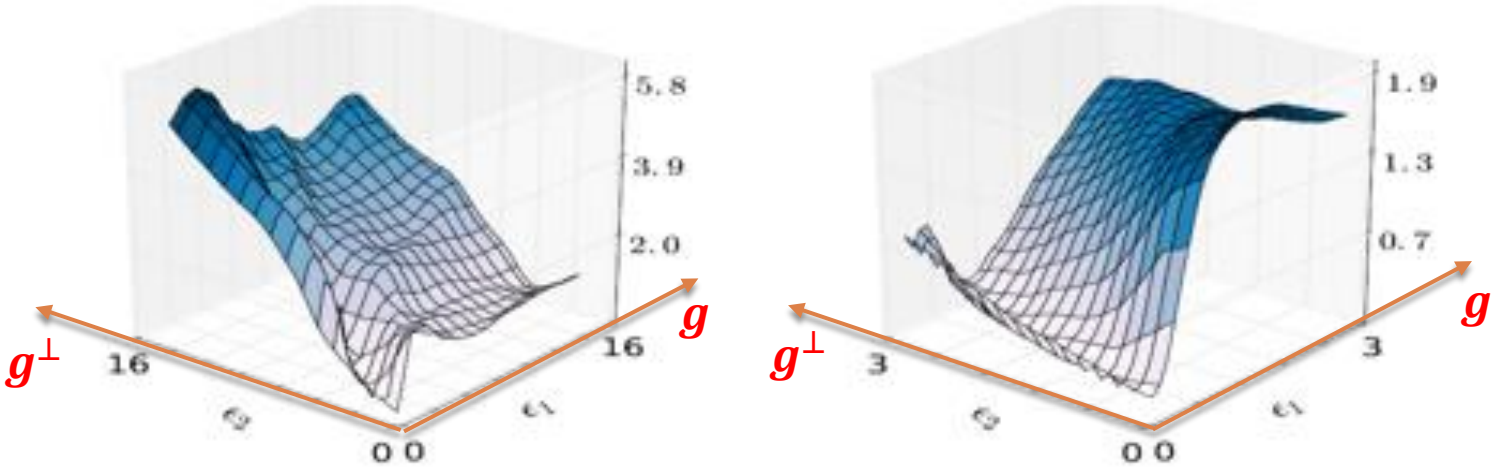
Decision regions of deep learning model



- Deep learning models are **fragile** and **easy to be attacked**
  - Start from $x = x + 0g + 0g^{\perp}$ (coordinate $(\mathbf{0}, \mathbf{0})$), if we go along with the gradient direction $g = \nabla_x l(f(x;\theta), y)$, we can reach $x_{adv} = x + \alpha g + 0g^{\perp}$ such that
    - x_{adv} is very close to x → human cannot tell how they **are different** → the **same class** from human **perceptron**
    - x_{adv} is predicted to **different class** from x by the classifier
    - x_{adv} is called **adversarial example** of x

Delving into Transferable Adversarial Examples and Black-box Attacks
Yanpei L., et al., ICLR 2016

Loss surface of deep learning model



Zoom out **Zoom in**

The **loss surface** of DL model **around** x. This is plotted on $x' = x + \epsilon_1 g + \epsilon_2 g^{\perp}$.

- At the **data point** x, if we move along with the **gradient direction** g, the **loss function** is **maximally locally increased**.
- **Taylor expansion** around x
  - $l(f(x+h;\theta),y) \approx l(f(x;\theta),y) + g^T h$ where $g = \nabla_x l(f(x;\theta),y)$.
  - The gradient direction is the **steepest direction** to increase the loss function locally.

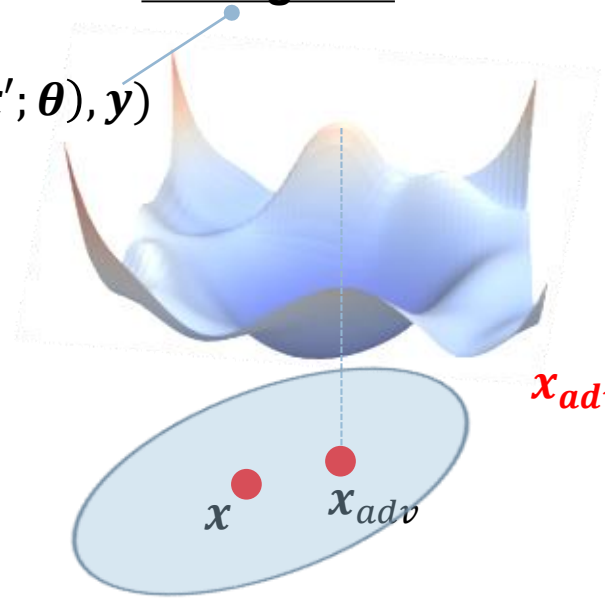
Ensemble Adversarial Training Attacks and Defences
Tramer et al., ICLR 2017

Adversarial attack

How to run untargeted attack?

***Untargeted* attacks**

Loss surface $\boldsymbol{l(f(x';\theta),y)}$



$B_\epsilon(x) = \{x' : \|x' - x\| \leq \epsilon\}$

$\boldsymbol{x}$ has true label $\boldsymbol{y} \in \{\mathbf{1}, \mathbf{2}, \ldots, \boldsymbol{M}\}$

$$\boldsymbol{x_{adv} = argmax_{x' \in B_\epsilon(x)} l(f(x';\theta),y)}$$

- Fast Gradient Sign Method (FGSM)
  - $x_{adv} = x + \epsilon sign(\nabla_x l(f(x;\theta),y))$
  - $sign(t) = \begin{cases} 1 & if\ t > 0 \\ -1 & if\ t < 0 \\ 0 & otherwise \end{cases}$
  - One-step update

Explaining and Harnessing Adversarial Examples
Goodfellow et al., ICLR 2015

- Projected Gradient Descent (Ascent) (PGD)
  - $x_0 = x + Uniform([-\epsilon, \epsilon])$
  - $\tilde{x}_{t+1} = x_t + \eta sign(\nabla_x l(f(x_t;\theta),y))$
  - $x_{t+1} = Proj_{B_\epsilon(x)}(\tilde{x}_{t+1})$
  - Run in k steps ($k = 20$), $\eta > 0$ is the learning rate
  - $x_{adv} = x_k$

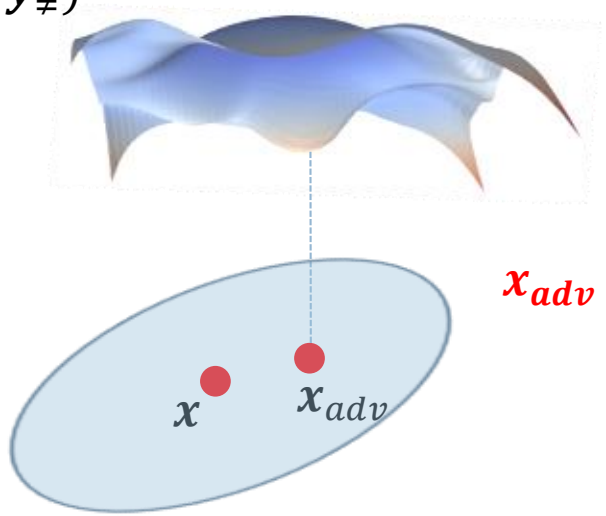
Towards Deep Learning Models Resistant to Adversarial Attacks
Madry et al., ICLR 2017

Adversarial attack

How to run targeted attack?

***Targeted* attacks**

Loss surface $\boldsymbol{l(f(x';\theta), y_{\neq})}$
$\boldsymbol{y_{\neq}}$ is different from **y**



$B_\epsilon(x) = \{x' : \|x' - x\| \leq \epsilon\}$
$\boldsymbol{x}$ has true label $\boldsymbol{y} \in \{\mathbf{1}, \mathbf{2}, \ldots, \boldsymbol{M}\}$

$$\boldsymbol{x_{adv} = argmin_{x' \in B_\epsilon(x)} l(f(x';\theta), y_{\neq})}$$

- Fast Gradient Sign Method (FGSM)
  - $x_{adv} = x - \epsilon sign(\nabla_x l(f(x;\theta), y_\#))$
  - $sign(t) = \begin{cases} 1 & if\ t > 0 \\ -1 & if\ t < 0 \\ 0 & otherwise \end{cases}$
  - One-step update

- Projected Gradient Descent (Ascent) (PGD)
  - $x_0 = x + Uniform([-\epsilon, \epsilon])$
  - $\tilde{x}_{t+1} = x_t - \eta sign(\nabla_x l(f(x_t;\theta), y_\#))$
  - $x_{t+1} = Proj_{B_\epsilon(x)}(\tilde{x}_{t+1})$
  - Run in k steps ($k = 20$), $\eta > 0$ is the learning rate
  - $x_{adv} = x_k$

Implementation of Attacks

```python
def fgsm_attack(model, input_image, input_label=None,
                epsilon=0.3,
                clip_value_min=0.,
                clip_value_max=1.0):
    """
    Args:
        model: pretrained model
        input_image: original (clean) input image (tensor)
        input_label: original label (tensor, categorical representation)
        epsilon: perturbation boundary
        clip_value_min, clip_value_max: range of valid input
        from_logits = True: attack from logits otherwise attack from prediction probabilities
    Note:
        we expect the output of model should be logits vector
    """

    if type(input_image) is np.ndarray:
        input_image = torch.tensor(input_image, requires_grad=True)

    if type(input_label) is np.ndarray:
        input_label = torch.tensor(input_label)

    # Ensure the model is in evaluation mode
    model.eval()

    # Create a copy of the input image and set it to require gradients
    adv_image = input_image.clone().detach().requires_grad_(True)  # Ensure requires_grad is True

    # Random initialization around input_image
    random_noise = torch.FloatTensor(input_image.shape).uniform_(-epsilon, epsilon)
    adv_image = adv_image + random_noise
    adv_image = torch.clamp(adv_image, clip_value_min, clip_value_max).detach().requires_grad_(True)

    output = model(adv_image)

    if input_label is not None:
        loss = F.cross_entropy(output, input_label)  # use ground-truth label to attack
    else:
        pred_label = output.argmax(dim=1)  # use predicted label to attack
        loss = F.cross_entropy(output, pred_label)

    model.zero_grad()
    loss.backward()
    gradient = input_image.grad.data
    adv_image = input_image + epsilon * gradient.sign()
    adv_image = torch.clamp(adv_image, clip_value_min, clip_value_max)

    return adv_image.detach()
```

FGSM attack

```python
def pgd_attack(model, input_image, input_label=None,
               epsilon=0.3,
               num_steps=20,
               step_size=0.01,
               clip_value_min=0.,
               clip_value_max=1.0):

    if type(input_image) is np.ndarray:
        input_image = torch.tensor(input_image, requires_grad=True)

    if type(input_label) is np.ndarray:
        input_label = torch.tensor(input_label)

    # Ensure the model is in evaluation mode
    model.eval()

    # Create a copy of the input image and set it to require gradients
    adv_image = input_image.clone().detach().requires_grad_(True)  # Ensure requires_grad is True

    # Random initialization around input_image
    random_noise = torch.FloatTensor(input_image.shape).uniform_(-epsilon, epsilon)
    adv_image = adv_image + random_noise
    adv_image = torch.clamp(adv_image, clip_value_min, clip_value_max).detach().requires_grad_(True)

    # If no input label is provided, use the model's prediction
    if input_label is None:
        output = model(adv_image)
        input_label = torch.argmax(output, dim=1)

    # Perform PGD attack
    for _ in range(num_steps):
        adv_image.requires_grad_(True)  # Ensure requires_grad is True in each iteration
        output = model(adv_image)
        loss = nn.CrossEntropyLoss()(output, input_label)
        model.zero_grad()
        loss.backward()

        # Check if gradient is available before accessing 'data'
        if adv_image.grad is not None:
            gradient = adv_image.grad.data
            adv_image = adv_image + step_size * gradient.sign()
            adv_image = torch.clamp(adv_image, input_image - epsilon, input_image + epsilon)  # Clip
            adv_image = adv_image.detach()  # Detach to prevent gradient accumulation
        else:
            print("Warning: Gradient is None. Check for detach operations.")

    return adv_image.detach()
```

PGD attack

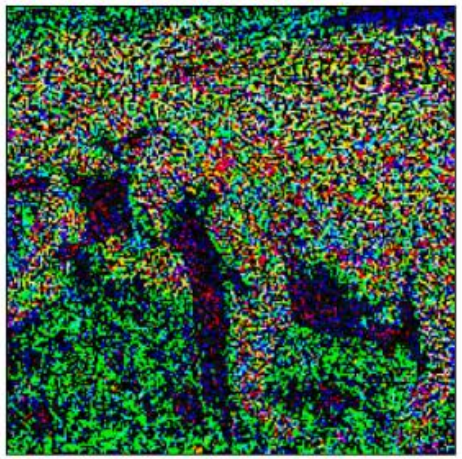
Adversarial Examples and Perturbations

```
attack_types = ['fgsm', 'trades', 'pgd']
attack_type = attack_types[2]

x_pgd = attack(attack_type, vgg19, batch_t, None, epsilon=0.01, num_steps=10, step_size=0.002, clip_value_min=-255.0, clip_value_max=255.0)
```



Original image: tusker



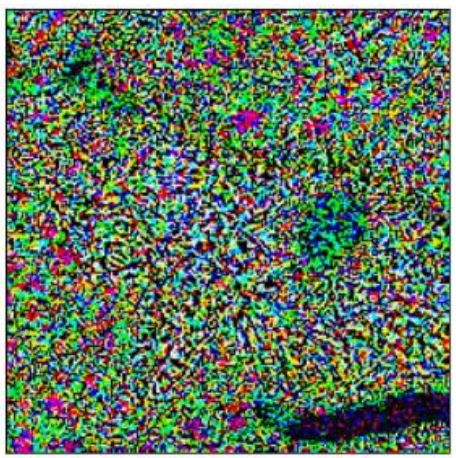
Noise



Adversarial image: African bush elephant



Original image: Siamese cat



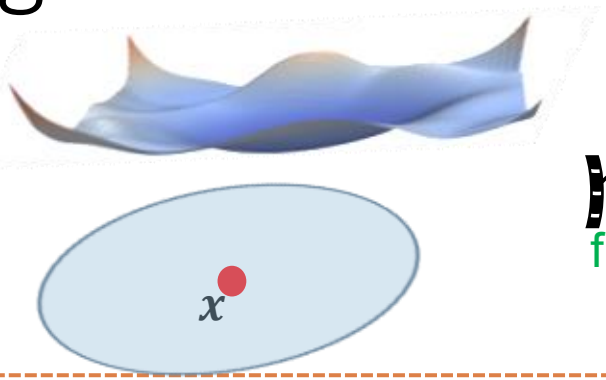
Noise



Adversarial image: borzoi

Adversarial training

Loss surface $l(f(x';\theta),y)$



We wish a **smooth** (or flat) **loss surface** around x

- The **ideal optimization problem** for adversarial training
  - $\min_{\theta} \mathbb{E}_{(x,y)\sim p(x,y)} \left[\max_{x' \in B_\epsilon(x)} l(f(x';\theta),y) \right]$
  - $p(x,y)$ is the **data distribution** that generates the pair of data point x and label y
  - Minimize the loss of over **most violated** adversarial examples

- **PGD adversarial training**
  - for epoch in n_epochs
    - for iter in range(n_iter_per_epoch)
      - Sample mini-batch $(x_1,y_1),\dots,(x_b,y_b)$ from the training set
      - Find PGD untargeted adversarial examples $x_1^{adv},\dots,x_b^{adv}$ for $x_1,\dots,x_b$ w.r.t labels $y_1,\dots,y_b$
      - $batch_loss = \frac{1}{b}\sum_{i=1}^{b} l(f(x_i;\theta),y_i) + \frac{1}{b}\sum_{i=1}^{b} l(f(x_i^{adv};\theta),y_i)$
      - $\theta = \theta - \eta \frac{\partial batch_loss}{\partial \theta}$

Adversarial training in practice

Principle to develop machine learning models

- Training condition **must match** testing condition

In reality, given $x_1, \dots, x_b$ and a model f_θ, how to **attack**?

↓

Use f_θ to predict $x_1, \dots, x_b$ → predicted labels $y_1^{pred}, \dots, y_b^{pred}$

↓

Attack f_θ using $y_1^{pred}, \dots, y_b^{pred}$ → **untargeted adversarial examples** $x_1^{adv}, \dots, x_b^{adv}$

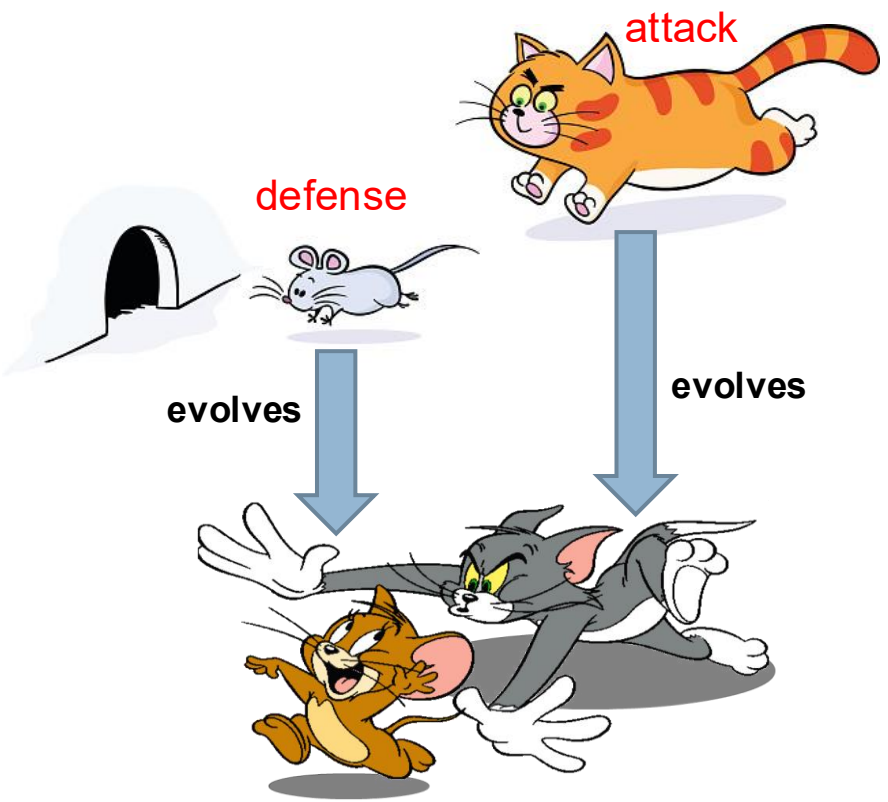
↓

Need to ensure f_θ to **secure** to these $x_1^{adv}, \dots, x_b^{adv}$?

PGD adversarial training in practice

- for epoch in n_epochs
  - for iter in range(n_iter_per_epoch)
    - Sample mini-batch $(x_1, y_1), \dots, (x_b, y_b)$ from the training set
    - Evaluate the predicted labels $y_1^{pred}, \dots, y_b^{pred}$ by the **current model**, i.e., $y_1^{pred} = argmax_m f_m(x_1; \theta)$, and etc.
    - Find PGD untargeted adversarial examples $x_1^{adv}, \dots, x_b^{adv}$ for $x_1, \dots, x_b$ w.r.t labels $y_1^{pred}, \dots, y_b^{pred}$
    - $batch_loss = \frac{1}{b}\sum_{i=1}^{b} l(f(x_i; \theta), y_i) + \frac{1}{b}\sum_{i=1}^{b} l(f(x_i^{adv}; \theta), y_i)$
    - $\theta = \theta - \eta \frac{\partial batch_loss}{\partial \theta}$

Attack vs Defense: Cat-and-mouse game



(Source: Internet)

- **Attack** is **stronger** than **defense**
- No defense method can **really defend** on ImageNet dataset.

Summary

- Deep learning before and after 2012
- Revision of building-blocks of CNN
  - Conv layer, pooling layer, global pooling layer
  - Drawback of CNNs
- SOTA Convolutional Neural Networks
  - GoogLeNet and ResNet
- Visualization in CNNs
  - Activation map, visualizing filters, last layer nearest neighbour
- Adversarial Machine Learning
  - Adversarial examples, attack and defense

Thanks for your attention!

